

CohortWatch

“This group is getting the same illness” — turning a fleet-wide firehose of weak vehicle signals into outbreak campaigns, a ranked workshop queue and approved actions.

100,000 simulated vehicles, 2 OEM formats	≤ 5 s event → campaign membership target	1 command docker compose up full stack + simulator	3 days solo build plan, MoSCoW- scoped
--	---	---	---

Problem space: Predictive maintenance / fleet reliability (Problem Statement §5) · Primary user: Fleet manager (reliability lead) · Secondary: workshop planner · Industry reference: Motorq (independent academic exercise, no affiliation) · Prepared 28 Sep 2026 · Working name “CohortWatch” — rename freely.

Contents

1	Executive summary	13	System design principles
2	Problem statement, users and scope	14	API and UI design
3	Evidence and industry analogues	15	Security and compliance
4	Solution overview and user journey	16	Observability
5	Architecture (HLD)	17	Testing strategy
6	Dataset and simulator design	18	DevOps and one-command setup
7	Kafka design	19	Edge-case catalogue
8	Database design (polyglot)	20	NFR targets and honesty rules
9	Detection and campaign algorithms	21	3-day build plan and feature list
10	Spark / Hadoop batch design	22	Demo video script
11	ML design and evaluation	23	ADRs, risks and future work
12	Agentic AI design	24	Deliverables traceability
		—	Sources

How to read this plan

Every section maps to a part of the Problem Statement (§6 Challenge, §7 Technical Expectations, §8–§13) or the Solution Document Template. Items are tagged with MoSCoW priority for a **3-day, one-person** build: **MUST** **SHOULD** **COULD** **WON'T** (documented only). Anything not built is labelled honestly in the feature table (Done / Partial / Planned) — the template expects that.

One product, one title

The title is the cohort campaign. OEM-format normalisation, reconnect/burst handling and privacy-safe sharing are built *underneath* it as plumbing; firmware generation is a clue inside the campaign. If a feature does not change **incident**, **membership** or **queue rank**, it is out of scope.

1. Executive summary

Problem. A fleet of 100,000 connected vehicles produces a constant stream of weak signals — a slightly hot engine, a battery draining a little faster, an intermittent fault code. Each one alone is cheap noise, so workshops fix whichever vehicle complains loudest. But the expensive failures are usually **shared**: the same model, at the same depot or on the same duty, on the same software generation, drifting away from its own normal in the same way. No single alert shows that; only the group does.

Solution. CohortWatch learns each vehicle's own normal (level *and* slope), raises an **incident** only when a vehicle leaves its own line after adjusting for what its peers in the same conditions are doing, and groups incidents into one **campaign** per shared “illness” (fault family × model × duty × place band × time window). New events add members in seconds; look-alike vehicles from a different context stay out. Vehicles that share the campaign profile but have not yet crossed their line are flagged as **at-risk sisters** — the predictive part. A ranked **workshop queue** puts outbreak members above loud-but-lonely warnings, and a guard-railed **agent** drafts inspection lists and asks before disruptive actions.

Why it fits the brief. The decision genuinely needs the whole fleet (an outbreak of 15 among 100,000 is invisible per vehicle or per depot), so the 100K scale is part of the problem rather than decoration. It exercises every expectation in §7: messy bursty ingestion, partitioned Kafka streams with replay, real-time detection plus Spark batch over a billion-row history, a relational core that truly needs ACID (campaign membership, approvals, audit), Redis for hot state, pgvector for “this looks like the pump failure last time”, an ML model evaluated against a baseline with planted ground truth, and an agent with approvals and an audit trail.

Headline targets (to be measured, not assumed)

Measure	Target	How
Planted outbreak membership precision / recall	≥ 0.95 / ≥ 0.90	Simulator ground-truth labels, back-test job
Decoys wrongly admitted	0	Decoy labels in ground truth
Detection lead vs “loudest-first” baseline	Earlier by $\geq 50\%$ of onset duration	Replay of planted scenarios
Event → board (critical)	p95 < 5 s; dashboard < 2 s	Board-lag metric (event ts vs SSE push)
Ingest throughput	Report the measured figure; design for 100K ev/s	Producer/consumer benchmark + consumer lag
API latency	p95 < 200 ms, p99 < 500 ms	k6

2. Problem statement, users and scope

2.1 In plain words

Think of a doctor noticing an outbreak. One coughing patient means nothing. Twelve coughing patients from the same street, all getting worse this week, all healthy before — that is one illness with one cause. CohortWatch does this for vehicles: it notices when a group of similar vans in the same situation is “getting the same illness”, keeps coincidental look-alikes out, and tells the workshop who to see first.

2.2 Template format

A fleet reliability lead needs a way to turn thousands of weak, messy vehicle signals into a small number of outbreak campaigns — these similar vehicles, in the same kind of place and work, are leaving their own normal in the same way — because **each warning alone looks cheap and noisy, the costly failure is usually shared, and the workshop books whoever is loudest**, which today costs **weeks of detection delay, repeat roadside breakdowns in the same cohort, and warranty and downtime cost that grows every day the shared cause stays unnamed** (industry: ~7 weeks from first report to confirmed root cause; warranty ≈ 2.2% of OEM sales, see §3).

Role	Who	What they need from the product
Primary user	Fleet manager — reliability lead	Campaign board: size, place, severity, getting worse or not; clues in plain phrases; override “not an outbreak”.
Secondary	Workshop planner	A ranked queue: who takes the next bay, why, and the rough cost of waiting.
Secondary	Depot operator (viewer)	Campaign summary for their depot, without anyone’s movement diary.
Downstream stakeholder	OEM quality team	An exportable campaign report (signature, firmware, members count) — a field-report feed, not a UI.
Not users	Bank recovery, city traffic team, insurer, driver-scoring	Explicitly out of scope; behaviour data is a clue, never a driver score.

2.3 Problems this architecture is for

1. **Alert flood with no story** — thousands of lights, no grouping.
2. **One company-wide threshold** (“too hot”) that is wrong for highway vs yard duty and for vehicles with different personal normals.
3. **Eight vans, eight tickets, one hidden cause** — batch, depot, supplier or software update.
4. **Workshop calendar as a shouting match** — loudest first, not most consequential first.
5. **Snapshot blindness** — a level (“92 °C”, “40%”) without the slope hides vans that are getting worse.
6. **False families** — same fault code, different worlds (different depot, duty or climate).
7. **Clones** — late or repeated notes that add the same van to an outbreak twice.

2.4 What the extra inputs may and may not change

Input	Plain meaning	May change	Must not become
Behaviour	Speed, harsh stops, idle, short-trip hammering	Whether the pattern is real for this kind of day; queue rank (capped weight)	A driver-punishment score
Slope of health	How fast temperature, charge, range, warning rate are moving	Urgency, clues, at-risk status	A research model with no queue
Place context	Depot, corridor band, shift, weather band, crowding	Who counts as a sister; context adjustment	A live traffic map
Shared context	Model, duty type, software generation	The family key; firmware clue	An OTA rollout-management product

2.5 Scope boundaries

In scope Per-vehicle baselines; incidents with clues; campaign formation, growth, merge and close; at-risk sisters; workshop queue; override; agent proposals with approval; audit; privacy roles and erasure; multi-OEM normalisation; burst/late/duplicate handling; batch baselines, back-test and fault-rate cube.	Out of scope (traps) Maps and live traffic; driver safety scoring; ten campaign types “for completeness”; OTA rollout control; parts inventory/ERP; claiming 100,000 events/s when only 100,000 IDs were created.
--	---

3. Evidence and industry analogues

3.1 Evidence and validation

Evidence / assumption	Source or method	What it shows	Confidence
Global automotive warranty cost ≈ \$58B (2024), ≈ 2.2% of sales; > 4% of revenue for some OEMs; nearly doubled since 2012	McKinsey (2025) survey	Size of the pain for shared field failures	High (published)
≈ 7 weeks from first issue report to confirmed root cause, plus 9–10 weeks to deploy countermeasures	McKinsey	Detection latency is the lever; faster grouping shortens it	High
Only 18% of executives run AI/ML in production to predict quality issues; 85% say data consolidation is early/intermediate	McKinsey	Gap between intent and capability — room for this product	High
Detection-to-correction cycles up to 200 days; delay can cost up to \$1M/day for a large issue	EE Times (telematics for warranty)	Telematics sees faults as they happen vs claims lag	Medium (trade article)
Regulators already expect aggregated field data: US TREAD Act Early Warning Reporting (quarterly warranty claims, field reports, complaints)	NHTSA EWR	Cohort-level evidence has a regulatory consumer	High
Current fleet AI (e.g. Motorq Fuse, Apr 2026) flags issues per vehicle with dollar impact	Motorq press release	Cohort/outbreak grouping is a differentiated layer	Medium
Assumption: 12–20 sisters, onset over 2–5 days, is a realistic outbreak shape	Simulation design	Drives window and threshold choices	Low — tuned by back-test
Assumption: a roadside breakdown costs ≈ 8–15× a planned inspection	Stated as parameter	Used only for rough “cost of waiting”	Low — configurable

3.2 Existing alternatives and why they fall short

Alternative	Falls short because
Aftermarket dongles + threshold alerts	Per-vehicle, fixed thresholds, no cohort; hardware to install.
OEM portals	One brand at a time; mixed fleets can’t see cross-brand context.
Warranty-claims analytics (OEM quality)	Starts after repair claims — weeks late; sees claims, not live signal.
Per-vehicle AI copilots	Explain one vehicle well but do not say “these 15 share one cause”.
Generic ML clustering on all telemetry	Unexplainable families (“cluster 7”), merges different worlds, fails the explain test.

3.3 Industry practices we borrow and implement

Practice	Where it comes from	How CohortWatch implements it	Priority
Early-warning / emerging-issue detection	OEM warranty early-warning systems; NHTSA EWR aggregation	Campaign = emerging issue; statistical guard that the cohort's incident rate is above its historical expected rate before opening.	MUST
Peer-group (context-aware) anomaly detection	Context-aware unsupervised fleet PdM (peer group of similar vehicles in the same context, J. Intell. Inf. Syst., 2022)	Context adjustment: a vehicle's deviation is measured against its own baseline <i>minus</i> the median deviation of its peers in the same place/weather/hour. A heatwave that warms everyone does not open a campaign.	MUST
Cohort analysis by software version	Fleet cyber-security cohort analysis (cohorts by OTA version, Upstream)	Firmware generation in the vehicle registry; clue “14/15 members updated to 4.2.1 within 3 days before onset”; optional firmware-keyed family.	SHOULD
Fault-log vectorisation + clustering	TF-IDF over vehicle-week fault logs + k-means (railway fleet study, 2026)	Campaign signature embedding (TF-IDF of fault codes ⊕ numeric clue profile) in pgvector for “like last time”; Spark MLlib k-means as offline family discovery.	SHOULD
Space-time outbreak scan statistics	Epidemiology (e.g. space-time scan statistics used for disease-cluster surveillance)	Poisson tail test on (family key, window): open only when $P(X \geq k \mid \lambda_{\text{expected}}) < \alpha$. Directly mirrors the “outbreak” metaphor and controls false campaigns.	MUST
Field-report export	Regulatory/early-warning reporting	CSV/JSON campaign report for the OEM quality team (no personal data).	COULD
Device-free OEM-cloud ingestion + normalisation	Motorq normalisation engine	Two OEM payload formats (and a mid-run v2 change) mapped by adapters to one canonical event.	SHOULD

4. Solution overview and user journey

4.1 Core objects

Object	Meaning	Speed / consistency
Vehicle	VIN, model, duty, home depot, firmware history	Reference data, ACID
Event	One note + OEM sequence number + event time	Fast, messy, AP
Latest state	The vehicle now	Redis, overwritten
Baseline	This vehicle's usual level and usual slope per metric (healthy windows only)	Batch-built, versioned
Incident	This vehicle crossed its own line + short clue list	Stream-created, idempotent
Campaign	The family: signature + place band + time window	Meeting-room record, CP
Member	This vehicle in that campaign (once); role = member or at-risk	CP, unique
Queue item	Who should take a bay, why, cost of waiting	CP
Proposal / Override / Audit line	Agent suggestion, human decision, who did what	CP, append-only audit

4.2 User journey (event → outcome)

1. Vehicles stream notes via two OEM feeds; the normaliser validates, deduplicates and converts them.
2. The state processor updates latest state and the vehicle's rolling slope, compares to its own baseline with context adjustment → **incident** with clues ("coolant +9 °C over its normal, rising 0.8 °C/day, 3× its usual slope, urban duty, D-042 heat band 3").
3. The campaign engine finds or opens the **campaign** for that family key; the vehicle joins once; at-risk sisters are recomputed; the workshop queue is re-ranked.
4. The board updates in < 2 s via server-sent events. The agent drafts "Inspect these 12 at D-042 tomorrow; 3 already in service tomorrow".
5. The reliability lead approves (audit line), or marks "not an outbreak" (sticky override). Workshop planner sees the queue move.
6. Nightly Spark jobs rebuild baselines on healthy windows only, refresh the fault-rate cube and embeddings, and run the back-test that proves the numbers.

4.3 Screens (three built, two cheap)

Screen	Contents	Priority
Campaign board	Cards: size, place, fault family, severity, trend arrow (growing/stable), firmware clue; workshop-queue side panel; agent proposal cards with Approve / Ignore; activity feed.	MUST
Campaign detail	Members + at-risk list; clues; shared context; "similar past campaigns" (vector); override history.	MUST
Vehicle vs its own normal	Metric chart with baseline band, slope, incident markers; location masked for viewer.	MUST
Board lag / pipeline health	Grafana: ingest rate, consumer lag, event→board p95.	SHOULD
Audit viewer	Filterable audit log (admin only).	COULD

4.4 What “done” looks like (becomes the BDD suite)

1. Plant on → a campaign opens on the sisters.
2. Decoy (same fault, different depot or duty) stays out; heatwave decoy opens nothing.
3. One more sister appears in the stream → member count +1, no second campaign.
4. Clues mention slope and shared place, not only a warning light.
5. Queue puts outbreak members above a loud but lonely warning.
6. Duplicate or late note does not clone a member; a late note lands in the right time window.
7. “Not an outbreak” override survives the next tick; re-proposal only if materially worse.
8. Viewer role cannot see precise location trails; every campaign view writes an audit line.
9. A stranger can start it with one command and retell the story in five minutes.

4.5 Innovative ideas (template §3.3)

Idea	What is new	Evidence planned
Self-baseline <i>and</i> peer-context deviation, on level and slope	Most fleet tools use global thresholds or per-vehicle only; we require “left its own normal more than its peers did”.	Heatwave decoy produces zero campaigns; global threshold produces a false campaign.
Explainable outbreak scan (rule-based family key + Poisson guard + union-find merge)	Grouping is a defensible rule, not an opaque cluster ID; statistics control false alarms.	Precision/recall on planted truth vs k-means clustering baseline.
At-risk sisters	Turns detection into prediction: same profile, not yet sick → inspect first.	Back-test: % of later incidents that were pre-flagged at-risk, lead time gained.

5. Architecture (high-level design)

5.1 System context (C4 level 1)

Actor / system	Relationship to CohortWatch	Protocol
OEM clouds (simulated OEM-A “Aurex”, OEM-B “Kestrel” — fictitious)	Push vehicle notes in their own formats (device-free model, like Motorq)	Kafka produce to raw topics (connector); HTTPS batch in production
Reliability lead, workshop planner, viewer	Use the board, approve/override, run the queue	HTTPS + SSE (browser)
Identity provider (Keycloak)	Issues OIDC tokens; roles and tenant claim	OIDC / OAuth2, JWKS
LLM API (optional, pluggable)	Writes the plain-language summary from structured evidence only	HTTPS
OEM quality team (downstream)	Receives exported campaign reports (no personal data)	File / webhook (COULD)

5.2 Container diagram (C4 level 2)

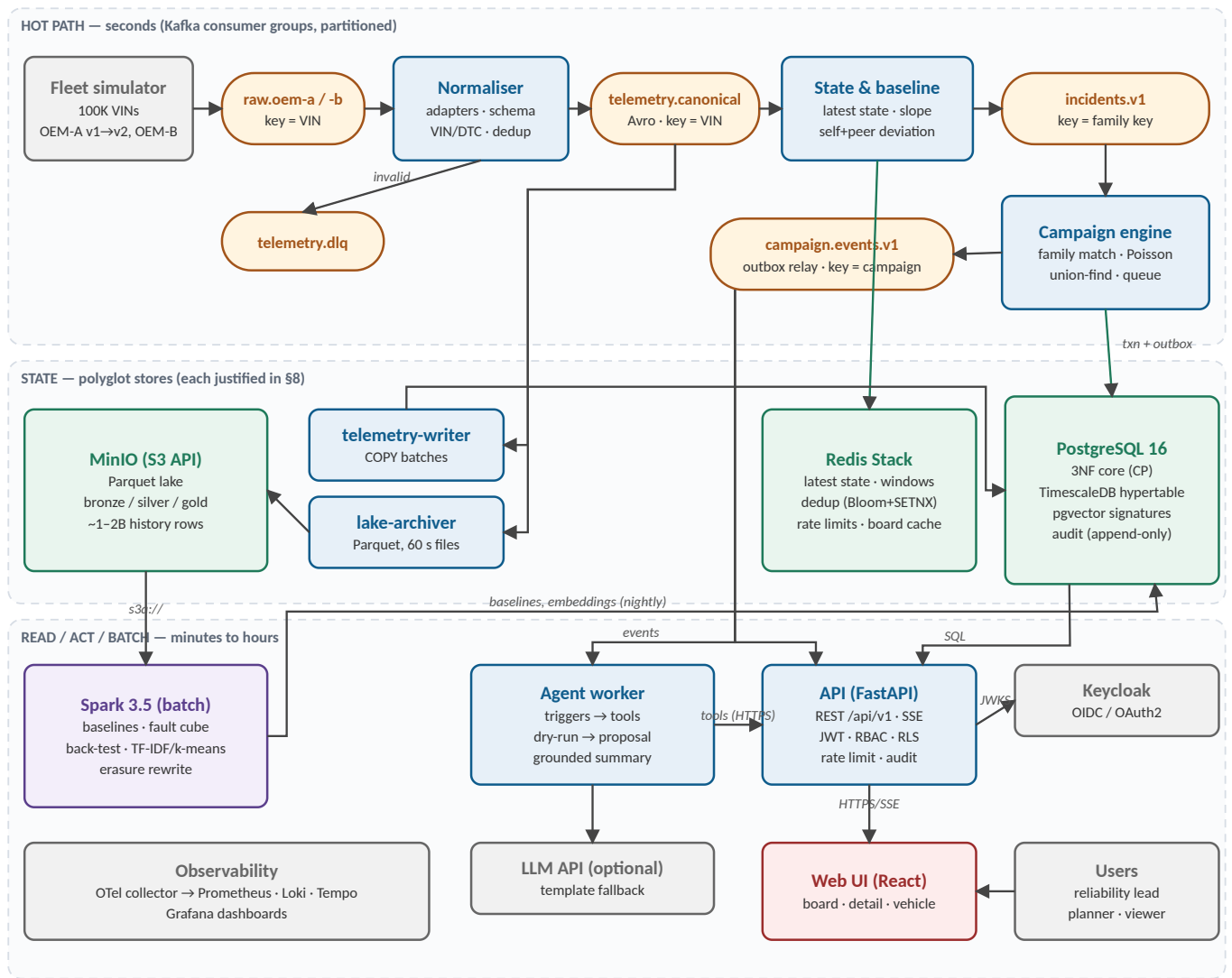


Figure 1 — Containers, stores and protocols. Solid arrows are the event path; every hot-path hop is a Kafka topic, so any service can be killed and replayed.

5.3 Services and responsibilities

Service	Responsibility	Scales by	Priority
simulator	100K-vehicle registry + event stream in two OEM formats; planted scenarios; noise injection; history generator; ground-truth labels.	processes / -- rate	MUST
normaliser	Adapter per OEM format/version; schema validation; VIN check-digit; DTC regex → fault family; unit conversion; dedup (anti-replay window); DLQ.	consumer instances ≤ raw partitions	MUST
state-processor	Latest state; O(1) exponentially weighted slope; self-baseline z-scores; peer-context adjustment; incident emission with clues; top-K rising codes (Count-Min).	≤ 48 instances (canonical partitions); state in Redis	MUST
campaign-engine	Family key; Poisson guard; open/grow/merge (union-find)/close; membership once; at-risk sisters; queue scoring; transactional outbox.	≤ incidents partitions; single writer per family	MUST
outbox-relay	Publishes committed campaign events from the outbox table to Kafka (at-least-once, idempotent keys).	1 active (leader lock)	MUST
telemetry-writer	Down-samples to 1 row/vehicle/minute and bulk-COPYs into Timescale.	instances	MUST
lake-archiver	Writes Parquet (bronze) to MinIO every 60 s per partition.	instances	SHOULD
api	REST + SSE; OIDC/JWT; RBAC + Postgres row-level security; rate limit; keyset pagination; audit on every read of personal data.	stateless replicas	MUST
agent	Trigger → tools → dry-run → proposal/auto polite action; grounded summary; audit of every tool call.	1–2 workers	SHOULD
web	React board, campaign detail, vehicle-vs-normal.	static (nginx)	MUST
spark jobs	Compaction, baselines, fault-rate cube, back-test, embeddings/k-means, erasure rewrite.	workers	SHOULD

5.4 Path of one telemetry event (latency budget)

#	Hop	Work	Budget (p95)
1	OEM feed → raw.oem-*	Producer batch (linger 5–10 ms), acks=all	30 ms
2	normaliser	Adapt, validate, dedup (Redis pipeline), produce canonical	80 ms
3	state-processor	Update state + slope sums, z-scores, context adjust (batched Redis round-trip)	150 ms
4	incidents.v1 → campaign-engine	Family lookup, Poisson guard, member upsert, queue re-rank, outbox (one Postgres txn)	250 ms
5	outbox-relay → campaign.events.v1	Poll/notify every 100 ms	150 ms
6	api → browser	Invalidate board cache, SSE push	100 ms
	Total	Event time → card on board	≈ 0.8 s typical; alarm if > 5 s

Under a 3× burst the budget is dominated by consumer lag; the board shows its own lag (“board is 3.2 s behind”) rather than pretending to be live.

5.5 Technology stack and justification

Layer	Choice	Why this, and what was rejected
Messaging	Redpanda (Kafka API) + built-in Schema Registry	Kafka protocol (portable to Apache Kafka/MSK/Confluent/Event Hubs), partitioned ordering, replay; single binary, no ZooKeeper — fits one-command setup. Rejected RabbitMQ (no log replay), MQTT broker as the spine (device-free OEM-cloud model; MQTT kept as a documented ingress option).
Stream processing	Python services on Kafka consumer groups (confluent-kafka)	State is per-vehicle and O(1) (exponentially weighted sums in Redis), so a heavyweight engine is not needed; one language end-to-end for a solo build. Rejected Flink/Kafka Streams (JVM ops cost in 3 days) — ADR-6 records when to switch.
Batch	Apache Spark 3.5 (PySpark) on MinIO (S3 API) Parquet	\$7 names Spark + HDFS/S3; s3a works unchanged on AWS S3/GCS/Azure-compatible gateways. Rejected HDFS (NameNode ops); DuckDB kept as a dev-speed fallback for the same SQL.
Relational (CP)	PostgreSQL 16 (timescaledb-ha image)	ACID for campaigns, membership uniqueness, approvals, audit; row-level security for tenant isolation.
Time-series	TimescaleDB hypertable + continuous aggregates	Time partitioning, native compression, “vehicle vs its own normal” queries in ms. Rejected Cassandra (extra cluster; our query is per-VIN time range, Timescale handles it).
NoSQL / cache (AP)	Redis Stack	Latest state, slope sums, anti-replay dedup, Bloom, Count-Min/Top-K, token-bucket rate limits, board cache. Rejected MongoDB (no need for documents; Redis covers the key-value hot path).
Vector	pgvector (HNSW) in Postgres	Signatures are joined to campaigns and filtered by tenant/RLS; one fewer store to secure. Qdrant rejected for now behind a <code>SignatureIndex</code> port (swappable).
Object store	MinIO	S3 API locally; same code on any cloud.
Backend / Frontend	FastAPI + SQLAlchemy 2 (async) · React + Vite + Recharts	Typed contracts (OpenAPI auto-generated), async SSE; small SPA.
Identity	Keycloak (realm import on start)	Real OIDC/OAuth2 flows and roles with no cloud dependency.
ML / AI	NumPy/scikit-learn/LightGBM, Spark MLlib; agent = small state machine (LangGraph optional) + any LLM via adapter	Deterministic policy decides; LLM only phrases. Works with no LLM key.
Observability	OpenTelemetry → Prometheus, Loki, Tempo → Grafana	Metrics, logs, traces in one pane; ELK rejected (memory footprint).
DevOps	Docker Compose, Helm, Terraform (AWS first), GitHub Actions	One command locally; same images and Helm chart on any Kubernetes.

6. Dataset and simulator design

The simulator is part of the solution (§8). Its job is to make the lie **consistent**: a large healthy population, a planted sister group, decoys that must stay out, and realistic mess — with a ground-truth file so every claim can be scored.

6.1 Fleet composition (registry, seeded to Postgres)

Dimension	Values (defaults, configurable)
Vehicles	100,000 VINs with valid ISO 3779 check digit; fictitious WMIs per OEM
Tenants	2 (Tenant A 70,000 vehicles; Tenant B 30,000) — proves tenant isolation
OEMs / formats	OEM-A “Aurex” (nested JSON, v1 °F → v2 °C switch mid-run); OEM-B “Kestrel” (flat compact JSON, epoch-ms, SoC as 0–1)
Models	8 models: EV vans, diesel vans, hybrid, EV compact, diesel truck
Powertrain mix	55% EV · 35% diesel · 10% hybrid
Depots	400 depots in 8 regions; each region has a climate profile (hot-dry, hot-humid, temperate, cold) and daily ambient curve
Duty types	Urban delivery 45% · linehaul 20% · rental 15% · field service 10% · yard shuttle 10% — each with speed, stop, idle and load profiles
Firmware	3–4 versions per model; staged rollout events recorded in <code>vehicle_firmware_history</code>
Drivers	Pseudonymous IDs with assignment history (enables erasure flow; never used for scoring)

6.2 Signals and healthy behaviour

Signal	Applies to	Healthy model (per vehicle)
<code>coolant_c</code>	Diesel, hybrid	88 + personal offset $N(0,2)$ + duty load- $k + 0.15 \cdot (\text{ambient} - 25)$ + warm-up curve + $N(0,0.8)$
<code>batt_temp_c</code>	EV, hybrid	ambient + 6 + load term + personal offset + noise; charging adds +4
<code>soc_pct</code> / <code>fuel_pct</code>	EV / ICE	Drain per km = model rate × personal efficiency × ambient penalty × behaviour factor; charges/refuels at depot
<code>lv_batt_v</code>	All	12.6 ± 0.15 resting, 14.1 ± 0.2 charging
<code>speed_kmh</code> , <code>odo_km</code> , <code>lat/lon</code>	All	Duty-specific trip generator around home depot; odometer monotonic
<code>harsh_brake</code> , <code>harsh_accel</code> , <code>idle_s</code>	All	Duty + personal style; clue only
<code>dtc[]</code>	All	Background Poisson $\lambda \approx 0.02$ codes/vehicle-day from the whole catalogue (the noise every fleet sees)
<code>ambient_c</code>	All	Region climate curve + weather events (heatwave)

6.3 Fault library (fault family ← DTC regex `^[PCBU][0-3][0-9A-F]{3}$`)

Fault family	Codes (SAE J2012 style)	Signal signature when sick
COOLING	P0217 engine over-temp, P0118 coolant temp sensor high, P0480 fan control circuit	Coolant level +6–12 °C over own normal; slope +0.5–1.2 °C/day; intermittent codes rising in frequency
HV_BATTERY_THERMAL	P0A7E battery pack over-temp, P0A80 replace battery pack	Battery temp +3–5 °C; drain +8–12%

LV_ELECTRICAL	P0562 system voltage low, P0620 generator control circuit	12 V resting voltage slope -0.05 V/day
EXHAUST_AFTERTREATMENT	P2463 DPF soot accumulation, P0401 EGR flow insufficient	Frequent regen; short-trip duty dependent
BRAKE_SENSOR	C0035 wheel speed sensor circuit	Code only (background family)

6.4 Planted scenarios and decoys (ground truth)

ID	Scenario	Must happen
S1	Cooling outbreak: 18 diesel vans, one model, urban duty, depot D-042 (hot-dry). Onset staggered over 3 sim-days. 15 cross their line during the run; 3 stay sub-threshold then cross late.	One campaign; 15 members; 3 flagged at-risk first, then join ("13th sister"); no second campaign.
S2	Firmware outbreak (stretch): 24 EV vans across 9 depots, battery temp and drain rise within 3 days of firmware 4.2.1.	Firmware clue "22/24 updated ≤ 3 days before onset"; optional firmware-keyed campaign.
D1	Same fault family, scattered: 6 vans with COOLING codes at 6 different depots/duties, independent onsets.	Incidents yes, campaign no.
D2	Heatwave: region R5, ~4,000 vans, ambient +8 °C for 2 days.	No campaign (peer-context adjustment).
D3	Loud lonely van: 40 codes/day across families.	Queue rank below S1 members.
D4	Sensor glitch: coolant 25 → 140 → 25 °C in seconds.	Impossible-slope rejection; no incident.
D5	Depot transfer: 2 S1 sisters move to D-051 mid-story.	Stay in S1 campaign; history not rewritten.
D6	Gentle driving day: 2 S1 sisters driven lightly.	Remain members (behaviour not in key).
D7	Two real outbreaks, same family, same week, two depots.	Two campaigns, not merged.

Ground truth is written to `data/ground_truth.parquet`: `vin`, `scenario_id`, `role` (`sister` | `at_risk` | `decoy` | `background`), `fault_family`, `onset_ts`. It is **never** read by the detection services — only by tests and the evaluation job.

6.5 Mess injection (every knob in `simulator.yaml`)

Mess	Default	Tests
Exact duplicates (same VIN, seq)	2%	Dedup, membership once
Out-of-order delivery (delay 1–600 s)	5%	Event-time windows, watermark
Reconnect burst (offline 30–180 sim-min, then backlog flush)	1% vehicles/hour	Late notes, burst handling
Shift-start surge	3× rate for 5 min	\$11 burst NFR
Malformed JSON / missing required field	0.1%	DLQ
Invalid VIN (bad check digit; I/O/Q)	0.05%	Regex + check digit
Unknown DTC format	0.05%	Parser tolerance
Clock skew ±90 s	0.5% vehicles	Skew guard, ingest-vs-event time
Impossible values (SoC 140%, negative speed, odometer going back)	0.02%	Physical-range validation
OEM-A schema change v1 → v2 (field renamed, °F → °C)	at sim day 1, 12:00	Adapter versioning, shape detection
Charge % that rises while driving	0.01%	Impossible-slope rule

6.6 Rates, clocks and volumes

Mode	Rate	Purpose
demo (default in compose)	100K vehicles, 1 note / vehicle / 10 s ≈ 10K ev/s ; sim clock 720× (1 wall-s = 12 sim-min, so a 3-day onset plays in ~6 min)	Laptop-friendly live story
bench	1 note / vehicle / s ≈ 100K ev/s , 3× surge for 5 min	\$11 throughput test (report measured number)
history	Written straight to Parquet by a Spark job reusing the same signal model: 7 days × 1 row / 15 min ≈ 67M rows (default, first boot); <code>--billion</code> : 30 days × 1 row / 2 min × 12 active h ≈ 1.08B rows (~20–27 GB)	Baselines, back-test, “batch over billions of rows”

Event time (`event_ts`) is simulated time; latency is always measured in wall time (`ingest_ts` → SSE push). The two clocks are never mixed.

6.7 Schemas

Canonical event (Avro, subject `telemetry.canonical.v1-value`, BACKWARD compatible)

```
{ "type": "record", "name": "TelemetryEvent", "namespace": "cohortwatch.v1", "fields": [
  { "name": "event_id", "type": "string", // uuid5(vin, seq) – deterministic, idempotent
  { "name": "vin", "type": "string", { "name": "tenant_id", "type": "string",
  { "name": "oem", "type": { "type": "enum", "name": "Oem", "symbols": ["AUREX", "KESTREL"] },
  { "name": "seq", "type": "long",
  { "name": "event_ts", "type": { "type": "long", "logicalType": "timestamp-millis" },
  { "name": "ingest_ts", "type": { "type": "long", "logicalType": "timestamp-millis" },
  { "name": "lat", "type": ["null", "double"], "default": null, { "name": "lon", "type":
["null", "double"], "default": null,
  { "name": "speed_kmh", "type": ["null", "float"], "default": null, { "name": "odo_km", "type":
["null", "double"], "default": null,
  { "name": "ambient_c", "type": ["null", "float"], "default": null,
  { "name": "coolant_c", "type": ["null", "float"], "default": null, { "name": "batt_temp_c", "type":
["null", "float"], "default": null,
  { "name": "soc_pct", "type": ["null", "float"], "default": null, { "name": "fuel_pct", "type":
["null", "float"], "default": null,
  { "name": "lv_batt_v", "type": ["null", "float"], "default": null,
  { "name": "harsh_brake", "type": "int", "default": 0, { "name": "harsh_accel", "type": "int", "default": 0,
  { "name": "idle_s", "type": "int", "default": 0,
  { "name": "dtc", "type": { "type": "array", "items": "string", "default": [] },
  { "name": "fault_families", "type": { "type": "array", "items": "string", "default": [] },
  { "name": "firmware", "type": ["null", "string"], "default": null,
  { "name": "source_format", "type": "string", // e.g. "aurex.v2"
  { "name": "quality_flags", "type": { "type": "array", "items": "string", "default": [] } ] }
```

OEM-A “Aurex” v1 (°F) → v2 (°C)

```
// v1
{ "vehicle": { "vin": "1AXVN2E47RA000123",
  "t": "2026-09-25T10:15:02.120Z", "n": 88412,
  "pos": { "la": 21.1702, "lo": 72.8311 },
  "eng": { "coolantTempF": 199.4, "rpm": 1850 },
  "elec": { "v12": 12.61, "spd": 64.2,
  "codes": ["P0217"], "sw": "D5-3.8.0" }
// v2 (from sim day 1 12:00)
{ ..., "engine": { "coolant": { "tempC": 93.0 },
... }
```

OEM-B “Kestrel” (flat, compact)

```
{ "id": "2KSCE3A19SK004410", "ts": 1790331302120,
  "sq": 1202, "g": [21.17, 72.83], "s": 64.2,
  "soc": 0.412, "bt": 31.5, "v": 12.58,
  "dtc": "P0A7E|C0035", "fw": "4.2.1",
  "hb": 1, "ha": 0, "idl": 30 }
```

Adapters live in `normaliser/adapters/{aurex_v1,aurex_v2,kestrel_v1}.py`. An unknown shape is detected by required-key fingerprint → DLQ + metric `unknown_shape_total{oem}` + alert.

7. Kafka design

7.1 Topics

Topic	Key	Part.	Retention / cleanup	Format	Producer → consumers
raw.oem-a.v1, raw.oem-b.v1	VIN	24	3 d, delete	raw JSON bytes	connector/simulator → normaliser
telemetry.canonical.v1	VIN	48	3 d, delete	Avro (registry)	normaliser → state-processor, telemetry-writer, lake-archiver
telemetry.dlq.v1	source topic	6	14 d	JSON envelope (error, offset, payload b64)	normaliser → dlq-replayer, ops
incidents.v1	family key	24	14 d	Avro	state-processor → campaign-engine, lake-archiver
campaign.events.v1	campaign_id	12	30 d (+ lake)	Avro (event-sourced)	outbox-relay → api (SSE), agent, lake-archiver
agent.proposals.v1	campaign_id	6	30 d	Avro	agent → api
audit.v1	tenant_id	6	90 d (+ lake, immutable)	Avro	all services → audit-sink (Postgres append-only)
vehicle.state.v1 COULD	VIN	48	compacted	Avro	state-processor → api (rebuildable read model)

Dev: replication factor 1 (single broker). ha profile / production: 3 brokers, RF = 3, min.insync.replicas = 2.

7.2 Keys, partitions and hot spots

- **VIN key** gives per-vehicle ordering and spreads 100K keys uniformly — no hot partitions on the telemetry path.
- **Sizing:** 100K ev/s × ~220 B Avro ≈ 22 MB/s canonical; a Python consumer sustains roughly 5–10K ev/s with batching, so 48 partitions allow up to 48 parallel state-processors (headroom for the 3× burst).
- **Incidents keyed by family key** (ADR-3): all incidents of one family land on one partition, so exactly one campaign-engine instance decides “open or join” for that family. That removes the race that would otherwise open two campaigns for the same outbreak, without distributed locks. Incident volume is small (hundreds/s at most), so a large family does not create a throughput hot spot.

7.3 Delivery semantics

Concern	Setting / approach
Producer durability	acks=all, enable.idempotence=true, compression=zstd, linger.ms=5–20, batch.size=256KB
Consumer commits	Manual commit after the batch is durably processed → at-least-once ; cooperative-sticky assignor; max.poll.records=5000
Effectively-once results	Idempotent consumers: deterministic event_id; anti-replay window per VIN; Postgres UNIQUE(campaign_id, vin); incident key (vin, family, window); outbox events carry event_id so the API/agent can drop repeats
Exactly-once option	Kafka transactions (read-process-write) for normaliser → canonical are a COULD ; the report explains why idempotency is preferred (cheaper, survives DB side effects)

Back-pressure	Consumers <code>pause()</code> partitions when the internal queue or DB write latency crosses a threshold and <code>resume()</code> after draining; Kafka retention absorbs the burst; producers block on full buffers; in Kubernetes KEDA scales on consumer lag
Replay	Reset a consumer group to a timestamp to rebuild Timescale, the lake, or all campaign read models from <code>campaign.events.v1</code>
Schema evolution	Registry compatibility BACKWARD on canonical; new optional fields with defaults; OEM format changes absorbed in adapters (raw topics keep bytes, so a fixed adapter can replay)
Poison messages	Never block a partition: validation failure → DLQ with offset; replay tool re-feeds after a fix

7.4 Consumer groups

`cg.normaliser`, `cg.state`, `cg.tswriter`, `cg.archiver`, `cg.campaign`, `cg.api-sse` (one per API replica, latest offset), `cg.agent`, `cg.audit-sink`. Each group is independently replayable; lag per group is exported to Prometheus.

8. Database design (polyglot)

8.1 Polyglot map and CAP choice

Data	Store	CAP / PACELC	Why
Tenants, fleets, depots, models, vehicles, firmware history, drivers	PostgreSQL (3NF)	CP · PC/EC	Reference integrity, RLS, joins
Incidents, campaigns, members, queue, proposals, overrides	PostgreSQL	CP · PC/EC	“One member once”, approvals and queue must never diverge; a campaign is a meeting-room record
Audit log	PostgreSQL append-only (+ lake copy)	CP	Legal evidence; hash-chained rows
Telemetry (1/min down-sample, 14 d) + hourly aggregates (13 months)	TimescaleDB	Single primary; read replicas AP-ish · EL	Per-VIN time-range queries, compression
Latest state, slope sums, dedup windows, Bloom, CMS/Top-K, rate limits, board cache	Redis Stack	AP · EL (latency first)	Rebuildable from Kafka; stale by seconds is acceptable
Campaign signatures (64-d)	pgvector (HNSW, cosine)	CP (same DB)	Joined to campaigns; tenant-filtered
Full-fidelity history	MinIO / S3 Parquet	AP, eventually consistent listing	Cheap, columnar, Spark-native

8.2 ER diagram (relational core, 3NF)

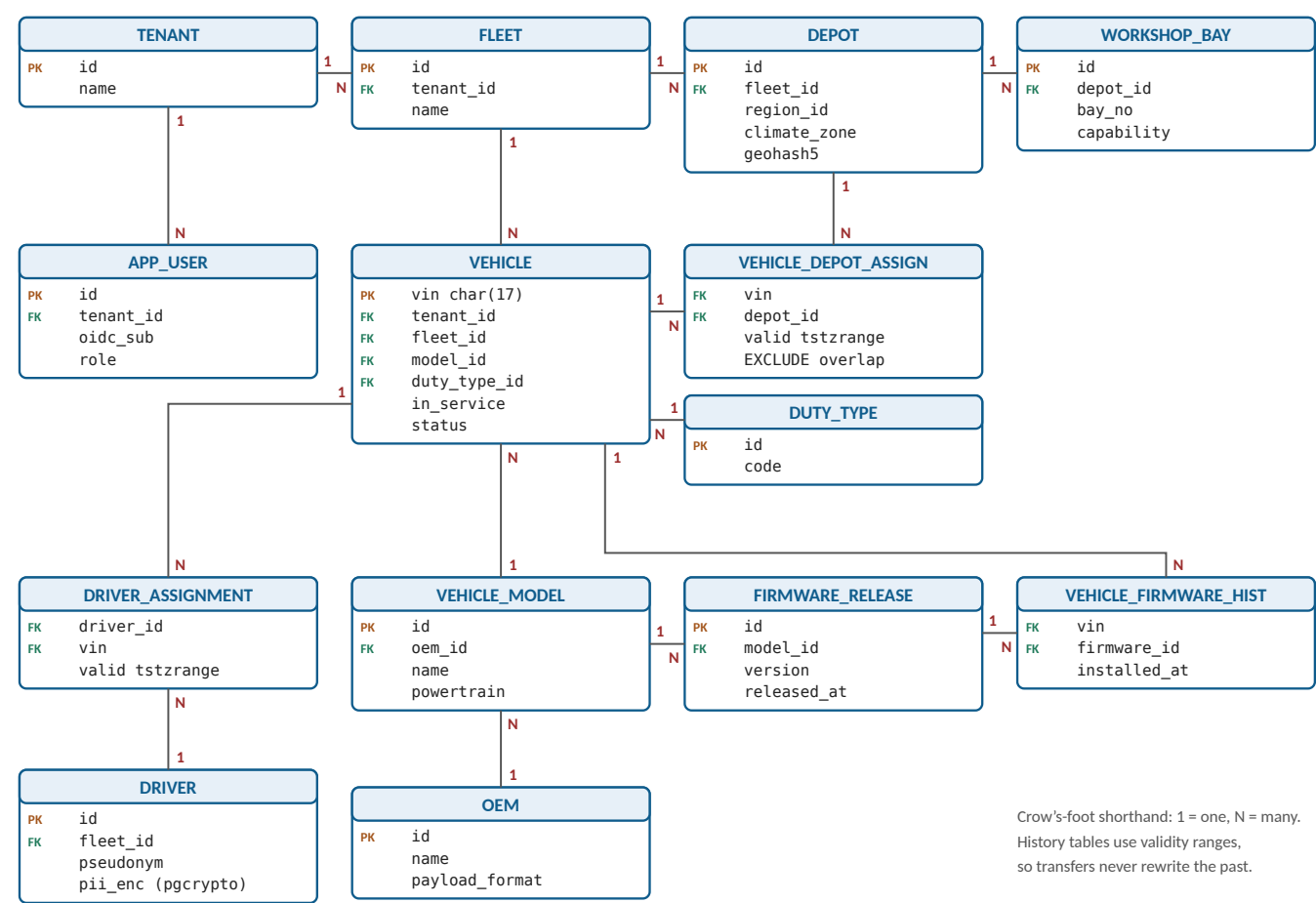
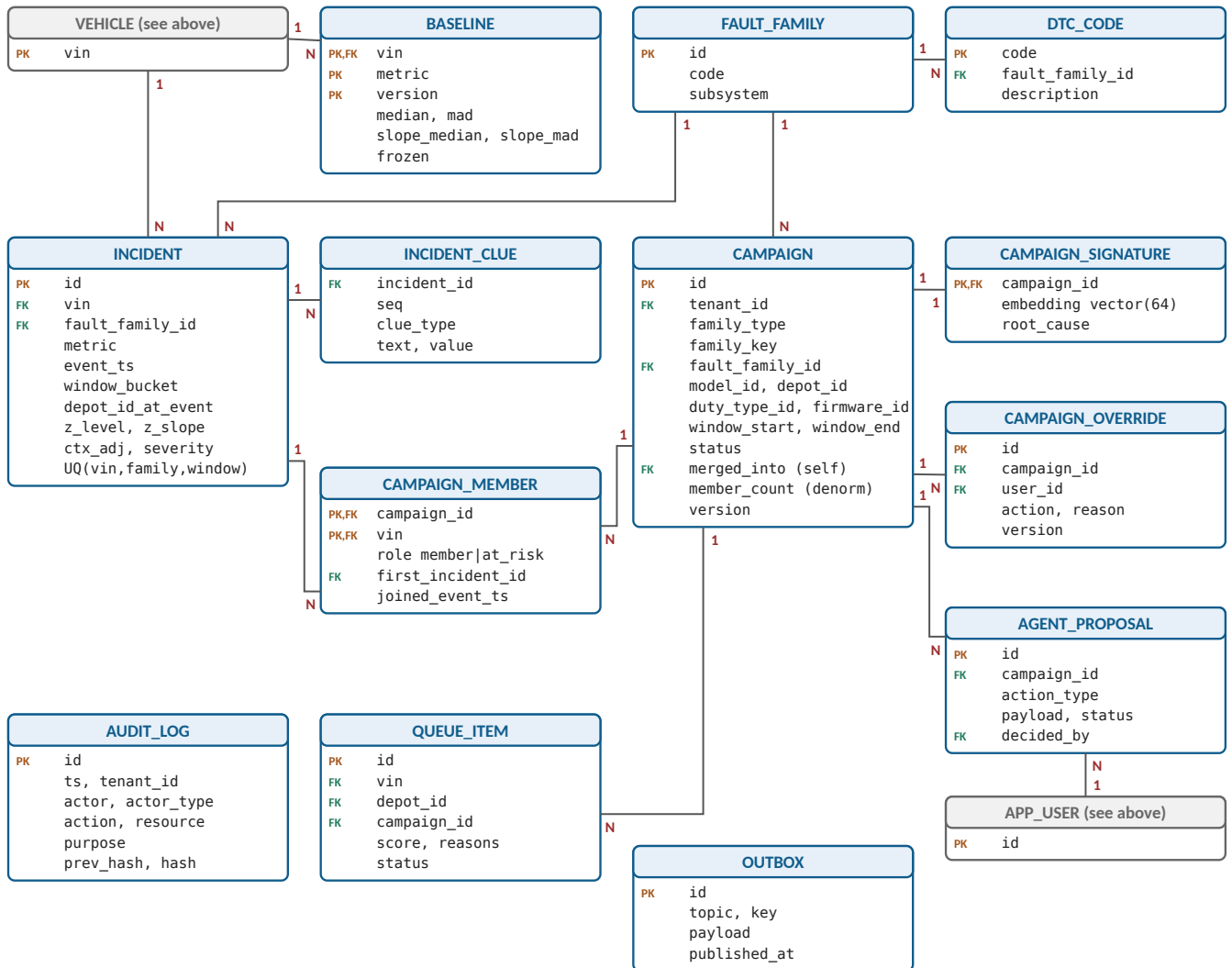


Figure 2a — Reference core (fleet, vehicles, history tables). Every non-key attribute depends on the key, the whole key and nothing but the key.



VIN foreign keys on MEMBER and QUEUE_ITEM reference VEHICLE (drawn once for clarity). OUTBOX rows are written in the same transaction as CAMPAIGN changes; AUDIT_LOG is append-only (INSERT only) and hash-chained. Timescale telemetry_1m and the continuous aggregate sit outside the 3NF core.

Figure 2b — Detection and decision core (incidents, campaigns, members, queue, proposals, audit).

8.3 Key DDL (abridged)

```
CREATE TABLE vehicle (
  vin          char(17) PRIMARY KEY CHECK (vin ~ '^[A-HJ-NPR-Z0-9]{17}$'),
  tenant_id    uuid NOT NULL REFERENCES tenant(id),
  fleet_id     uuid NOT NULL REFERENCES fleet(id),
  model_id     int  NOT NULL REFERENCES vehicle_model(id),
  duty_type_id int  NOT NULL REFERENCES duty_type(id),
  in_service   date NOT NULL);

CREATE TABLE vehicle_depot_assignment (
  vin char(17) REFERENCES vehicle(vin), depot_id int REFERENCES depot(id),
  valid tstzrange NOT NULL,
  EXCLUDE USING gist (vin WITH =, valid WITH &&)); -- no overlapping assignments

CREATE TABLE incident (
  id uuid PRIMARY KEY, tenant_id uuid NOT NULL, vin char(17) NOT NULL REFERENCES vehicle(vin),
  fault_family_id int NOT NULL REFERENCES fault_family(id),
  metric text NOT NULL, event_ts timestamptz NOT NULL,
  window_bucket timestamptz NOT NULL,
  depot_id_at_event int NOT NULL, model_id_at_event int NOT NULL,
  z_level real, z_slope real, ctx_adj real, severity real,
  UNIQUE (vin, fault_family_id, window_bucket)); -- two incidents, one window → one row
```

```

CREATE TABLE campaign (
  id uuid PRIMARY KEY, tenant_id uuid NOT NULL,
  family_type text NOT NULL CHECK (family_type IN ('place','firmware')),
  family_key text NOT NULL, -- e.g. COOLING|m7|URBAN|D-042
  fault_family_id int NOT NULL, model_id int, depot_id int, duty_type_id int, firmware_id int,
  window_start timestamptz NOT NULL, window_end timestamptz NOT NULL,
  status text NOT NULL CHECK (status IN
('candidate','open','confirmed','not_outbreak','closed','merged')),
  merged_into uuid REFERENCES campaign(id),
  member_count int NOT NULL DEFAULT 0, severity real, trend text,
  version int NOT NULL DEFAULT 0); -- optimistic concurrency for UI edits
CREATE INDEX campaign_open_idx ON campaign (tenant_id, severity DESC, id) WHERE status IN
('open','confirmed');

CREATE TABLE campaign_member (
  campaign_id uuid REFERENCES campaign(id), vin char(17) REFERENCES vehicle(vin),
  role text NOT NULL CHECK (role IN ('member','at_risk')),
  first_incident_id uuid REFERENCES incident(id), joined_event_ts timestamptz NOT NULL,
  PRIMARY KEY (campaign_id, vin)); -- the "no clones" guarantee

CREATE TABLE queue_item (
  id uuid PRIMARY KEY, tenant_id uuid NOT NULL, depot_id int NOT NULL, vin char(17) NOT NULL,
  campaign_id uuid, score real NOT NULL, reasons jsonb NOT NULL, cost_of_wait numeric(10,2),
  status text NOT NULL DEFAULT 'waiting');
CREATE UNIQUE INDEX queue_one_waiting ON queue_item (vin) WHERE status = 'waiting'; -- one open slot
per van
CREATE INDEX queue_waiting_idx ON queue_item (depot_id, score DESC, id) WHERE status = 'waiting';

CREATE TABLE audit_log (
  id bigserial PRIMARY KEY, ts timestamptz NOT NULL DEFAULT now(), tenant_id uuid,
  actor text NOT NULL, actor_type text NOT NULL, action text NOT NULL,
  resource_type text, resource_id text, purpose text, details jsonb,
  prev_hash bytea, hash bytea NOT NULL); -- hash = sha256(prev_hash || row) → tamper evidence

CREATE TABLE outbox (id uuid PRIMARY KEY, topic text, key text, payload bytea,
  created_at timestamptz DEFAULT now(), published_at timestamptz);

-- Timescale
CREATE TABLE telemetry_1m (vin char(17), ts timestamptz, coolant_c real, batt_temp_c real,
  soc_pct real, lv_batt_v real, speed_kmh real, ambient_c real, dtc_count smallint, geohash5 text);
SELECT create_hypertable('telemetry_1m','ts', chunk_time_interval => interval '1 day');
ALTER TABLE telemetry_1m SET (timescaledb.compress, timescaledb.compress_segmentby='vin');
SELECT add_compression_policy('telemetry_1m', interval '2 days');
SELECT add_retention_policy('telemetry_1m', interval '14 days');
CREATE MATERIALIZED VIEW vehicle_hourly WITH (timescaledb.continuous) AS
  SELECT vin, time_bucket('1 hour', ts) b, avg(coolant_c), max(coolant_c), avg(batt_temp_c),
    min(lv_batt_v), sum(dtc_count) FROM telemetry_1m GROUP BY vin, b;

-- Vector
CREATE TABLE campaign_signature (campaign_id uuid PRIMARY KEY REFERENCES campaign(id),
  tenant_id uuid NOT NULL, embedding vector(64) NOT NULL, root_cause text, resolution text);
CREATE INDEX ON campaign_signature USING hnsw (embedding vector_cosine_ops) WITH (m=16,
ef_construction=64);

ALTER TABLE campaign ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_iso ON campaign USING (tenant_id = current_setting('app.tenant_id')::uuid);

```

8.4 Deliberate denormalisation

Denormalisation	Why	How kept correct
incident.depot_id_at_event, model_id_at_event	History must reflect where the vehicle was <i>then</i> (transfer edge case); avoids a range join on every read	Written once, immutable
campaign.member_count, severity, trend	Board query without GROUP BY over members	Updated in the same transaction as member insert; nightly consistency check

<code>queue_item.reasons</code> (jsonb)	Explanations are rendered, not queried	Rebuilt on every re-rank
Board read model in Redis	CQRS read side	Invalidated by <code>campaign.events</code>

8.5 Redis key design

Key	Type	Content / purpose	TTL
<code>vs:{vin}</code>	Hash	Latest state (masked fields for viewers are removed at API level)	7 d
<code>sl:{vin}:{metric}</code>	Hash	Exponentially weighted regression sums S_w , S_{wt} , S_{wy} , S_{wtt} , S_{wty} + last t — $O(1)$ slope state	7 d
<code>ar:{vin}</code>	String (bitmap)	Anti-replay window: highest seq + 1,024-bit bitmap of recent seqs	7 d
<code>bf:seen:{day}</code>	Bloom (RedisBloom)	(vin, seq) older than the bitmap window; error rate 0.1%	3 d
<code>base:{vin}</code>	Hash	Baseline per metric: median, MAD, slope median, slope MAD, version, frozen flag	none
<code>ctx:{region}:{duty}:{hour}</code>	Hash / t-digest	Peer-context deviation (robust centre) for context adjustment	48 h
<code>cms:dtc:{hour}</code> , <code>topk:dtc</code>	Count-Min / Top-K	Rising fault codes fleet-wide (board ticker)	48 h
<code>rl:{sub}</code>	String	Token bucket (Lua) per user / client	1 min
<code>board:{tenant}</code>	String (JSON)	Cached board read model	invalidate on event

8.6 Capacity estimate and data lifecycle

Item	At design load (100K ev/s)	Hot / warm / cold
Raw JSON on Kafka	~0.7–1 KB/event → 70–100 MB/s → ~6–8.6 TB/day	3 d retention (× RF 3 in prod)
Canonical Avro	~220 B/event → ~22 MB/s → ~1.9 TB/day	3 d
Timescale 1-min down-sample	100K × 1,440 = 144M rows/day; ~10–20× compression → ~2–4 GB/day	Hot 14 d; hourly aggregates 13 months
Lake silver Parquet (full rate)	8.64B rows/day × ~10–14 B → ~90–120 GB/day	Warm 90 d S3 Standard; cold IA to 1 y; archive to 3 y; then delete
Postgres core	Campaigns/incidents: MBs/day; audit ~1–2 GB/month	Hot; audit partitioned monthly
Rough storage cost	90 d warm ≈ 9–11 TB ≈ \$210–250/month at S3 Standard list price (\$0.023/GB-month); archive tiers ~20× cheaper	Estimate only — state as such in the doc

8.7 SQL optimisation plan (EXPLAIN ANALYZE before/after)

Query	Before (naïve)	After
Q1 Campaign board (open campaigns, member count, joined-last-24h, top clue)	ORM loads campaigns then counts members per campaign (N+1); OFFSET pagination	Denormalised counters + partial index <code>campaign_open_idx</code> ; single query; keyset pagination on (severity, id)
Q2 Vehicle vs own normal, 7 days	Scan raw hypertable rows for the VIN	Continuous aggregate <code>vehicle_hourly</code> , index (vin, b)
Q3 Workshop queue top-20 for a depot	Sort all queue rows	Partial index <code>queue_waiting_idx</code> (depot, score DESC) — index-only top-k

Q4 At-risk sister search (same model, depot, duty; not member; z_slope > 1)	Join vehicle × latest state in SQL	Composite index vehicle(model_id, duty_type_id) + current-assignment partial index; slope pulled from Redis in one pipeline
Q5 Audit by resource and time	Seq scan	BRIN on ts + B-tree (resource_type, resource_id); monthly partitions

Plans and timings are committed to docs/sql/ and summarised in the solution document's table (before ms / after ms / change).

9. Detection and campaign algorithms

Rule of the design: a model may decide “is this vehicle abnormal versus itself?”; grouping stays a rule you can defend.

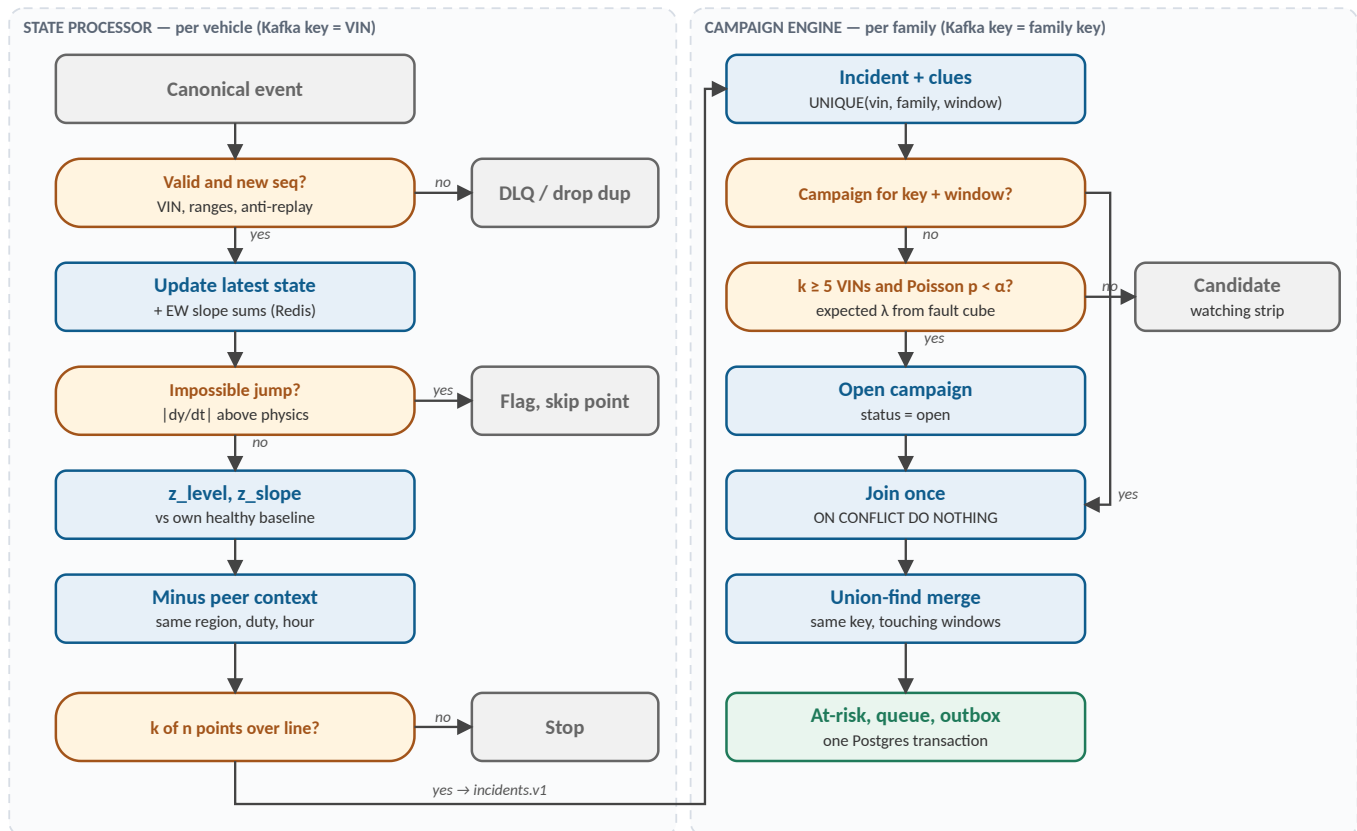


Figure 3 — Path of one note from event to campaign.

9.1 Validation and parsing

- **VIN:** regex `^[A-HJ-NPR-Z0-9]{17}$` (no I, O, Q) + ISO 3779 check digit (position 9: transliterate letters, weights 8,7,6,5,4,3,2,10,0,9,8,7,6,5,4,3,2; sum mod 11, 10 → “X”). $O(17)$.
- **DTC:** `^[PCBU][0-3][0-9A-F]{3}$`; family lookup by exact code, then by prefix table (e.g. `P0A7x` → `HV_BATTERY_THERMAL`). Unknown valid codes → family `OTHER`, never dropped.
- **Physical ranges:** SoC $\in [0,100]$, speed $\in [0,200]$, coolant $\in [-40,130]$; odometer monotonic per VIN; out-of-range → quality flag, field nulled, event kept.
- **Units:** adapter converts ($^{\circ}\text{F} \rightarrow ^{\circ}\text{C}$, 0–1 → %, epoch → timestamp); canonical is SI only.

9.2 De-duplication: anti-replay window + Bloom filter

```

def is_new(vin, seq):
    hi, bits = AR[vin]
    if seq > hi: shift bits by (seq-hi); set bit 0; hi = seq; return True
    d = hi - seq
    if d >= 1024: return not BF.EXISTS(day, vin:seq) and BF.ADD(...) # very late: Bloom
    if bits[d]: return False # exact duplicate inside window
    bits[d] = 1; return True # late but new (out-of-order)
  
```

Same idea as the anti-replay window in network security protocols. Exact inside the window; beyond it a Bloom filter (0.1% false positive) — a false positive routes the event to a *late path* (lake only) rather than silently dropping it. Downstream uniqueness constraints are the final guard.

9.3 Slope in O(1): exponentially weighted least squares

For each (VIN, metric) keep decayed sums; on a new point (t, y) after gap Δt , multiply all sums by $w = \exp(-\Delta t/\tau)$ ($\tau = 12$ sim-hours) then add the point:

```
S = w·S + 1      St = w·St + t      Sy = w·Sy + y
Stt= w·Stt+ t²    Sty = w·Sty + t·y
slope = (S·Sty - St·Sy) / (S·Stt - St²)    level_ewma = Sy / S
```

Five floats per metric ($\approx 100K \times 4 \times 5 \times 8 B \approx 16$ MB in Redis), so the state processors stay stateless and scale horizontally. Before adding a point, **impossible-slope guard**: if $|\Delta y/\Delta t|$ exceeds the metric's physical maximum (e.g. coolant > 5 °C/s, SoC rising while speed > 0), the point is flagged and skipped (decoy D4, “charge rises while driving”). Time t is **event time**; out-of-order points within 10 minutes are applied (sums are order-insensitive apart from decay, error bounded); later points go to the late path.

9.4 Self-deviation and peer-context adjustment

```
z_level = (level_ewma - base.median) / (1.4826 · base.MAD)      # robust z
z_slope = (slope - base.slope_median) / (1.4826 · base.slope_MAD)
ctx      = robust_centre( z_level of peers in (region, duty, hour) ) # excluding this VIN, t-digest median
adj_lvl = z_level - ctx ; adj_slp = z_slope - ctx_slope
abnormal_point = adj_lvl ≥ 3 or (adj_slp ≥ 3 and adj_lvl ≥ 1.5) or family_code_rate ≥ 4× own baseline
incident      = abnormal_point in ≥ 4 of the last 6 points      # short window kills one-tick noise
```

The peer-context term is the industry “peer group in the same context” idea: in a heatwave everyone's `z_level` rises, so `ctx` rises and `adj_lvl` does not (decoy D2). A single company-wide threshold fails exactly here — that is the baseline the evaluation compares against.

9.5 Incident clues (explain test)

Clues are generated from numbers into short phrases, never a bare score: “coolant 9.4 °C above its own normal”, “rising 0.8 °C/day — 3.1× its usual slope”, “P0217 seen 5× in 24 h (usual: 0)”, “peers at D-042 are normal today (context +0.2σ)”, “urban duty, hot-dry band”. Each clue stores its source numbers so the agent and UI can cite them.

9.6 Family key and time windows

```
place_band = depot_id          (urban, yard, rental, field)
            | region_corridor_id (linehaul – vans share roads, not yards)
family_key  = fault_family | model_id | duty_class | place_band      # type 'place'
firmware_key= fault_family | model_id | firmware_id                 # type 'firmware' (stretch)
bucket     = floor(event_ts / 24h) # event time, not wall clock
```

Behaviour is **not** in the key (a sick van driven gently must not fall out, D6). Weather band is a context adjuster and a clue, not a key (otherwise a cooler day splits a real outbreak).

9.7 Opening a campaign: count threshold + Poisson outbreak test

```
k      = distinct VINs with incidents for key in [t-72h, t]
N      = vehicles currently in the cohort (model × duty × place_band)
λ      = base_rate(family, model, duty) · N · 3 days # from Spark gold fault_rate_cube
p      = P(X ≥ k | Poisson λ) = regularised_gamma_P(k, λ)
open if k ≥ k_min (5) and p < α (1e-4, Bonferroni over keys evaluated per day)
else   status = 'candidate' (visible in a "watching" strip, no alert)
```

Scan-statistic thinking from disease-cluster surveillance: an outbreak is “more cases than expected in this place and time”. Decoy D1 (six scattered coolant incidents) never shares a key; a genuinely common fault in a big cohort needs more cases before opening than in a small cohort.

9.8 Merging split campaigns: union-find

```
parent = {} # campaign_id → parent id (persisted as merged_into)
def find(c): root = c; while parent[root] != root: root = parent[root]
    while parent[c] != root: parent[c], c = root, parent[c] # path compression
    return root
def union(a, b): ra, rb = find(a), find(b); if ra == rb: return
    keep older (or larger by rank) as root; mark other 'merged'
    INSERT members of child INTO root ON CONFLICT DO NOTHING
merge rule: same family_key AND windows overlap or touch (gap ≤ 24 h)
never merge: different place_band (D7: two depots, same week → two campaigns)
```

Amortised $O(\alpha(n))$ per operation. Because incidents are partitioned by family key, only one engine instance can union two campaigns of the same key — no cross-instance race.

9.9 Membership, at-risk sisters and lifecycle

- **Join once:** INSERT ... ON CONFLICT (campaign_id, vin) DO NOTHING; a van with two incidents in one window is one member.
- **At-risk sisters** (prediction): cohort vehicles that are not members with $\text{adj_slp} \geq 1.5$ or $\text{adj_lvl} \geq 2$ on the campaign's metric → role `at_risk`. Re-evaluated on every campaign change and every 10 sim-minutes. Scored against ground truth: “% of later members that were at-risk first” and lead time gained.
- **Transfer** (D5): membership is keyed to the campaign, not the current depot; incident rows keep `depot_id_at_event`.
- **Lifecycle:** candidate → open → confirmed | not_outbreak → closed (no new member for 72 sim-h and $\geq 80\%$ members inspected). “Not an outbreak” is sticky: suppressed for that key until members grow $\geq 50\%$ or severity rises $\geq 1\sigma$ (“materially worse”).

9.10 Workshop queue ranking

```
score = 0.35·severity_n + 0.25·slope_n + 0.20·campaign_factor + 0.10·in_service_tomorrow +
0.10·min(behaviour_burn, 1)
campaign_factor = log2(1 + members) / log2(1 + 20) # 0 for lonely vans
cost_of_wait = P(fail within 72h) · breakdown_cost - inspection_cost # rough, configurable
per-depot top-k via partial index (DB) and a heap for re-rank batches:  $O(\log n)$  insert,  $O(k)$  read
```

Decoy D3 (40 codes/day, no cohort, flat slope) ranks below S1 members: many lights ≠ high consequence. Weights live in config and are reported; a CI test pins the D3-below-S1 ordering.

9.11 Fleet-wide streaming structures

- **Count-Min Sketch + Top-K** (RedisBloom) of DTCs per hour: $\epsilon = 0.001$, $\delta = 0.01$ → width 2,719, depth 5; the board ticker shows “rising codes” = hour/24h-average ratio for top-K. $O(d)$ update, fixed memory.
- **t-digest** per context bucket for the peer median ($O(\log c)$ update, ~KB memory).
- **Geohash** precision 5 (≈ 4.9 km cells) for masked location; precision 7 only for the lead with a stated purpose.

9.12 Complexity summary (to be benchmarked in [docs/algorithms.md](#))

Function	Time	Space	Scale test
VIN check digit, DTC parse	$O(1)$ per event	$O(1)$	1M events micro-bench
Anti-replay dedup	$O(1)$	128 B/VIN (12.8 MB total)	100K VINs, 2% dups, 5% OOO
EW slope update	$O(1)$	40 B/metric/VIN	400K series
Peer context (t-digest)	$O(\log c)$	$O(\text{buckets} \cdot c)$	3,200 buckets

Poisson tail	$O(1)$ (regularised gamma)	$O(1)$	—
Union-find merge	$O(\alpha(n))$ amortised	$O(\text{campaigns})$	10K synthetic campaigns
At-risk search	$O(\text{cohort})$ per change	$O(\text{cohort})$	$\text{cohort} \leq 2,000$
Queue top-k	$O(\log n)$ insert, $O(k)$ read	$O(n)$	100K items
Count-Min / Top-K	$O(d)$ per code	$O(w \cdot d)$	fleet stream
HNSW similar-campaign search	$\sim O(\log n)$	$O(n \cdot m)$	1K–100K signatures

10. Spark / Hadoop batch design

Spark runs on the S3 API (MinIO locally; AWS S3, GCS or Azure via s3a/connector config) instead of HDFS — the “Hadoop” layer is the Hadoop S3A filesystem and Parquet, without NameNode operations (ADR-5). Compose starts a Spark master + 1 worker under the `batch` profile; jobs are submitted by `make batch` or a lightweight scheduler container (cron) in `full` profile.

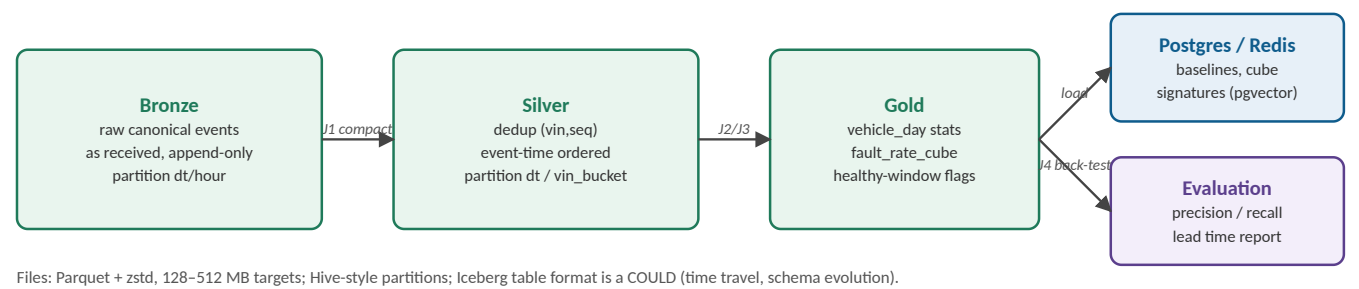


Figure 4 — Medallion layout on the object store.

10.1 Lake layout

```
s3://cohortwatch-lake/
bronze/telemetry/dt=2026-09-25/hour=10/part-*.parquet      # as received (canonical), append-only
bronze/incidents/... , bronze/campaign_events/... , bronze/audit/...
silver/telemetry/dt=.../vin_bucket=00..63/*.parquet      # dedup (vin,seq), event-time sorted,
compacted 256 MB
gold/vehicle_day/dt=.../                                # per VIN per day: robust stats, slopes,
dtc counts, healthy flag
gold/fault_rate_cube/                                    # (family, model, duty, place_band,
firmware, week) → rate
gold/eval/run_id=.../                                    # back-test results
history/                                                  # generated 100M / 1B-row history
labels/ground_truth.parquet                             # read only by eval job
```

10.2 Jobs

Job	Input → output	Key logic	Schedule	Priority
J0 history generator	registry → history/	Same signal model as the simulator (shared Python package, vectorised with pandas UDFs); includes 40 resolved historical campaigns with root-cause labels	once (seed)	MUST
J1 compaction	bronze → silver	<code>dropDuplicates(["vin", "seq"])</code> , event-time ordering, late-data merge, small-file compaction, <code>vin_bucket = hash(vin) % 64</code>	hourly	SHOULD
J2 baseline builder	silver/history → gold/vehicle_day → Postgres baseline + Redis base: *	Per VIN × metric: median, MAD, slope median/MAD over the last 14 days of healthy windows only (exclude days with incidents, campaign membership, or impossible-slope flags); if < 5 healthy days keep previous version (frozen)	nightly	MUST
J3 fault-rate cube	silver + incidents → gold/fault_rate_cube → Postgres	Incidents per 1,000 vehicle-days by family × model × duty × place_band × firmware × week; provides λ for the Poisson test and the batch-analytics dashboard	nightly	MUST
J4 back-test / evaluation	history + labels → gold/eval + report	Replays detection rules over history (same library as the stream); computes precision, recall, time-to-detect, at-risk lead time, decoy admissions, vs baselines	on demand + CI (small)	MUST

J5 signatures + discovery	incidents/campaigns → pgvector; vehicle-week DTC docs → k-means	TF-IDF over fault codes (HashingTF 256 → IDF) + SVD to 48 d ⊕ 16-d numeric clue profile = 64 d, L2-normalised; MLlib k-means (k by silhouette) over vehicle-weeks to suggest new fault families (offline, advisory)	nightly	SHOULD
J6 erasure / retention	erasure requests → rewrite affected partitions	Rewrite silver/bronze partitions without the VIN/driver rows (or with pseudonymised IDs); gold aggregates untouched (no personal data) so monthly totals do not change; delete-after-retention by lifecycle rules	daily	SHOULD

10.3 Spark configuration notes

- `spark.sql.shuffle.partitions` sized to data (e.g. 64 locally, 400 at 1B rows); AQE on; `spark.sql.files.maxPartitionBytes=256m`.
- s3a: `fs.s3a.endpoint`, `path.style.access=true`, credentials from env/secret — no code change between MinIO and a cloud bucket.
- Partition pruning on `dt`; bucketing by `vin_bucket` avoids shuffles in per-VIN jobs (J2).
- Benchmark to report: rows scanned, wall time, executors/cores, for J2 and J4 at 100M rows (default) and 1B rows (if hardware allows) — stated honestly.

11. ML design and evaluation

11.1 Purpose — why rules alone are not enough

A fixed threshold cannot know that van A always runs at 92 °C and van B at 84 °C, or that today is a heatwave. The ML layer answers one question — “**is this vehicle abnormal versus itself, given its context?**” — and a second, predictive one — “**which not-yet-sick sisters will cross their line in the next 72 hours?**”. Grouping and actions stay rule-based and explainable.

11.2 Models

Model	What it is	Why	Priority
M1 Robust self-baseline + peer context	Per-VIN robust statistics (median/MAD of level and slope) learned in Spark on healthy windows; streaming EW slope; peer-context correction (§9.4)	Transparent, O(1) streaming, no training labels needed; fits 100K per-vehicle models without per-vehicle training jobs	MUST
M2 At-risk classifier	LightGBM (or logistic regression fallback) predicting “becomes a campaign member within 72 h” from: adj z-level, adj z-slope, slope acceleration, family-code rate, cohort campaign size, days since firmware change, duty, age, ambient band	Turns detection into prediction; evaluable against the rule-based at-risk flag	SHOULD
M3 Signature embedding	TF-IDF(fault codes) ⊕ numeric clue profile → 64-d vector in pgvector	“Like the pump failure last time”; retrieval, not generation	SHOULD
M4 Family discovery	Spark MLlib k-means over vehicle-week fault profiles	Advisory only — suggests families for humans to name; never auto-creates campaigns	COULD

11.3 Data and leakage checks

- Training/evaluation data comes from the J0 history (with labelled historical outbreaks) and planted live scenarios; ground truth is only read by J4 and tests.
- **Time-based split** (train on days 1–20, validate 21–25, test 26–30) and **grouped by VIN** so a vehicle never appears in train and test.
- Features use only data with `event_ts ≤ prediction time` (point-in-time joins in Spark); no post-repair or “member” flags as features.
- Baselines exclude sick windows (frozen-baseline rule), otherwise the model learns that dying is normal.

11.4 Evaluation against simple baselines

Question	Our method	Baseline(s)	Metrics
Is this vehicle abnormal?	M1 (self + peer)	B1 company-wide fixed threshold (e.g. coolant > 100 °C); B2 self-baseline without peer context	Incident precision/recall vs ground truth; false incidents during heatwave D2
Is this an outbreak, and who is in it?	Family key + Poisson + union-find	B3 “loudest-first” (top-N DTC counts); B4 k-means on raw features	Membership precision/recall, decoys admitted, time-to-detect, number of campaigns (fragmentation)
Who is next?	M2 at-risk / rule at-risk	B5 random cohort member; B3	PR-AUC, precision@k for queue, mean lead time gained (hours)
Like last time?	M3 vector search	B6 same-family-code lookup	Top-3 root-cause hit rate on held-out historical campaigns

All metrics are produced by J4 into `docs/evaluation.md` and a Grafana panel; a small version runs in CI and fails the build if S1 recall < 0.9 or any decoy is admitted.

11.5 Operating the models

- Baselines and M2 are versioned (`baseline.version`, model artefact hash in `model_registry` table); incidents record the version used.
- Drift check: weekly distribution of adj z-scores per cohort; if the fleet median shifts $> 0.5\sigma$ without incidents, baselines are flagged for review rather than silently relearned.
- Cost: M1 is arithmetic in the stream; M2 scored in batches of affected cohorts (< 5 ms per 1,000 rows on CPU).

12. Agentic AI design

A junior agent that lives **on the board**, not in a chat tab. It watches triggers, gathers evidence with read-only tools, simulates one change, then either applies a polite action or raises a proposal card for a human. It may not invent clues and must ask before anything disruptive.

12.1 Loop

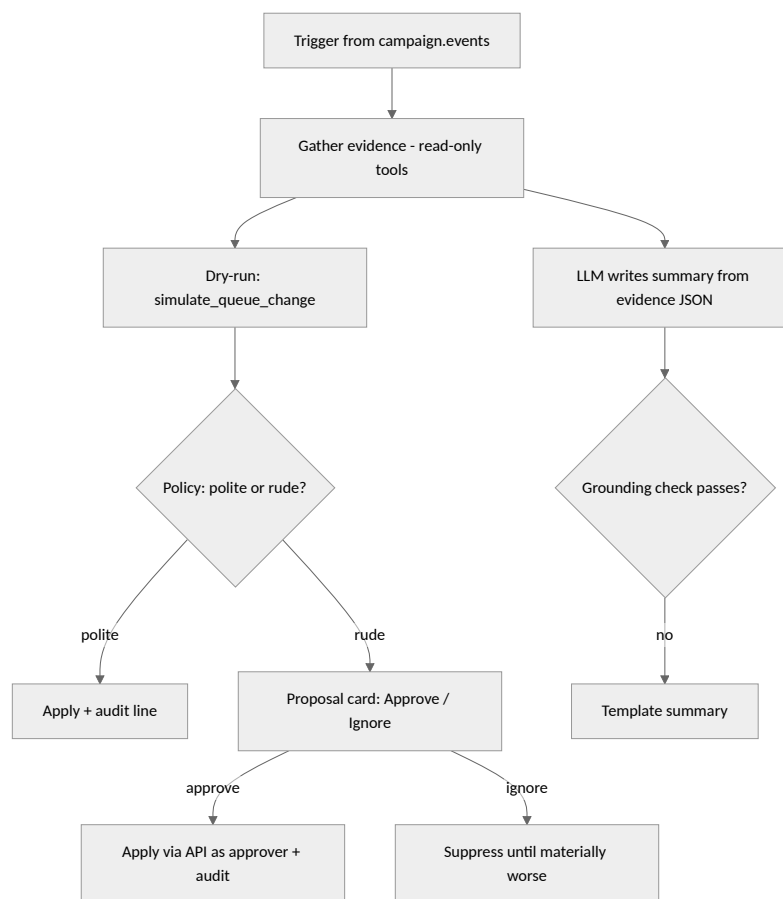


Figure 5 — Agent loop. The policy decides; the LLM only phrases.

12.2 Triggers

CampaignOpened · CampaignGrew ($\geq +25\%$ members or severity $+1\sigma$) · AtRiskSurge (≥ 5 new at-risk in a cohort) · QueueOverload (waiting $>$ bay capacity $\times 2$ for a depot) · FirmwareClue ($\geq 60\%$ members share a firmware change ≤ 3 days before onset).

12.3 Tools (typed, allow-listed; MCP-style)

Tool	Kind	Notes
<code>get_campaign(id), list_members(id, role)</code>	read	Masked location (geohash5 only)
<code>vehicle_vs_normal(vin, metric, days)</code>	read	Aggregates only; no raw trails
<code>search_similar_campaigns(id, k)</code>	read	pgvector; returns root causes + outcomes
<code>workshop_capacity(depot, date)</code>	read	Bays and bookings
<code>simulate_queue_change(action)</code>	dry-run	Returns new ranks, displaced vans, cost delta — no writes
<code>tag_watchlist(vins), draft_inspection_list(id), hold_tentative_bay(depot, slot)</code>	polite write	Auto-applied; reversible; audited
<code>propose(action, payload, evidence)</code>	proposal	For rude actions: pull van off tomorrow's route, page the whole workshop, hold manual firmware installs at our depots, notify OEM quality

12.4 Guardrails

- **Least privilege:** the agent's service account has scopes `campaign:read queue:simulate proposal:create watchlist:write` — never `proposal:approve`. Approval is a human-only endpoint.
- **Grounding check:** every number, VIN, depot and code in the LLM's text must appear in the evidence JSON; otherwise the template summary is used. No free prose reaches the board unchecked.
- **Prompt-injection defence:** canonical events contain no free text (only enums and numbers); OEM free-text fields are dropped by adapters; evidence is passed as data in a delimited block; the tool list is fixed per run.
- **Anti-nag:** max 1 open proposal per campaign per action type; an Ignored proposal is re-proposed only when materially worse (\$9.9); daily proposal budget per depot. Prevents “every event a card”.
- **Fairness:** “pull off route” proposals rotate across equally ranked vans; the same van is not repeatedly chosen.
- **Human override wins:** overrides carry a `version`; the agent reads the latest version before acting and never reverts a human decision.
- **Kill switch and degradation:** `AGENT_ENABLED=false` or LLM outage → board, queue and manual actions work unchanged; summaries fall back to templates.
- **Audit:** each run records `agent_run_id`, trigger, tool calls (name, args hash, result hash, latency), proposal, and who approved/ignored — linked in `audit_log`.

12.5 Evaluation and cost

Metric	Target
Correct action proposed for S1/S2	100% (scenario suite)
Proposals raised for decoys	0
Grounding check pass rate (LLM mode)	Reported; any failure falls back safely
Proposals per campaign per day	≤ 2 (anti-nag)
Latency / cost per run	Template mode ≈ 50 ms, \$0; LLM mode reported per provider (one short call, ~1–2K tokens)

13. System design principles

13.1 CAP / PACELC by data type

Data	On partition (CAP)	Else (PACELC)	Consequence
Campaign, membership, approvals, audit	Consistency (reject writes if primary unreachable)	Consistency over latency	Board may be read-only during a DB failover; never two campaigns or double approvals
Telemetry, latest state, windows	Availability (keep ingesting to Kafka)	Latency over consistency	Board may be seconds stale; it shows its lag
Board read model, caches	Availability	Latency	Rebuilt from events
Lake	Availability	Latency	Batch reads eventually complete data

13.2 Principles and where they live

Principle	Where / how
Partitioning & sharding	Kafka by VIN / family key; Timescale by time; lake by dt + vin_bucket; Postgres audit by month
Replication	Kafka RF 3 (ha profile/prod); Postgres streaming replica (Helm); MinIO erasure coding (prod)
Idempotency	Deterministic event_id; anti-replay; unique constraints; outbox event ids; idempotency keys on POST approvals
At-least-once vs exactly-once	At-least-once transport + idempotent effects (§7.3)
Back-pressure	Consumer pause/resume; bounded queues; HTTP 429 with Retry-After from API; KEDA lag scaling
Circuit breakers / retries	API → Postgres/Redis, agent → LLM, writers → DB: exponential back-off with jitter, breaker opens after N failures, half-open probes
CQRS	Write model: campaign-engine + Postgres. Read model: board cache + SSE from campaign.events
Event sourcing	campaign.events.v1 is the history of every campaign (Opened, MemberJoined, Merged, Overridden, Closed); read models are rebuildable by replay
Transactional outbox	State change + event in one Postgres transaction; relay publishes — no dual-write inconsistency
Cache invalidation	Event-driven (campaign event → delete board:{tenant}), plus short TTL as a safety net
Optimistic concurrency	campaign.version: two clicks on one campaign → second gets 409 and re-reads
Horizontal scaling	Stateless services; state in Redis/Postgres/Kafka; HPA/KEDA
Graceful degradation	LLM down → templates; Redis down → dedup falls back to DB constraints, board served from DB; Spark down → last baselines kept; agent down → manual actions
Hexagonal architecture	Domain (baseline math, family rules, queue scoring) has no framework imports; ports for broker, stores, LLM, signature index
12-factor	Config via env; logs to stdout; disposable processes; dev/prod parity with the same images

13.3 Layering (core service)

Layer	Responsibility	Must not
Presentation / API	HTTP, SSE, validation (Pydantic), auth checks, DTOs	Contain business rules or SQL
Application	Use cases: HandleIncident, ApproveProposal, MarkNotOutbreak; transactions	Depend on a specific DB/broker
Domain	Baseline, Incident, Campaign, FamilyKey, QueueScore, UnionFind, PoissonGuard	Import framework/infrastructure code

Infrastructure	Kafka, Postgres repositories, Redis, MinIO, LLM client	Leak vendor types into domain
----------------	--	-------------------------------

13.4 Design patterns

Pattern	Problem solved	Location
Adapter	Each OEM payload/version → canonical event	services/normaliser/adapters/
Strategy	Pluggable family-key types (place, firmware) and queue scoring	libs/domain/family.py, queue.py
Repository	Persistence behind interfaces	*/infra/repos.py
Transactional Outbox	Atomic state + event	campaign_engine/outbox.py
Circuit Breaker / Retry	Slow dependencies	libs/resilience.py
Observer / pub-sub	SSE fan-out, agent triggers	api/sse.py, agent/triggers.py
Chain of Responsibility	Validation pipeline (schema → VIN → ranges → dedup)	normaliser/pipeline.py
CQRS / Event Sourcing	Board reads vs campaign writes	campaign_engine/, api/readmodel.py

14. API and UI design

14.1 API contract (OpenAPI 3.1, `/api/v1`)

Method & path	Purpose	Roles
GET <code>/campaigns?status=&depot=&cursor=&limit=</code>	Board list (keyset pagination on severity, id)	all
GET <code>/campaigns/{id}</code>	Detail: clues, shared context, firmware clue, trend	all
GET <code>/campaigns/{id}/members?role=member at_risk&cursor=</code>	Members and at-risk sisters	all (masked for viewer)
GET <code>/campaigns/{id}/similar?k=3</code>	Vector search over past campaigns	lead, planner
POST <code>/campaigns/{id}/override {action: confirm not_outbreak close, reason, version}</code>	Human decision (optimistic concurrency → 409)	lead
GET <code>/queue?depot=&cursor=</code> , PATCH <code>/queue/{id}</code>	Workshop queue; planner can pin/defer	planner, lead
GET <code>/vehicles/{vin}/health?metric=&days=</code>	Vehicle vs its own normal (aggregates)	all; precise location only with purpose for lead
GET <code>/proposals?status=</code> , POST <code>/proposals/{id}/approve ignore</code> (Idempotency-Key)	Agent cards	lead (approve), all (read)
GET <code>/stream/board</code> (SSE)	Live board events with Last-Event-ID resume	all
POST <code>/privacy/erasure</code> , GET <code>/privacy/erasure/{id}</code>	Right-to-erasure workflow	privacy_admin
GET <code>/audit?resource=&from=&cursor=</code>	Audit search	auditor/admin
GET <code>/ops/board-lag</code> , <code>/healthz</code> , <code>/readyz</code> , <code>/metrics</code>	Ops	internal

Conventions: errors as RFC 9457 problem+json; rate limit 120 req/min/user (token bucket, 429 + `Retry-After`, `RateLimit-*` headers); max page size 100; ETag on detail; versioning by path; AsyncAPI document for Kafka topics in `docs/asyncapi.yaml`.

14.2 Runtime flows (sequence diagrams)

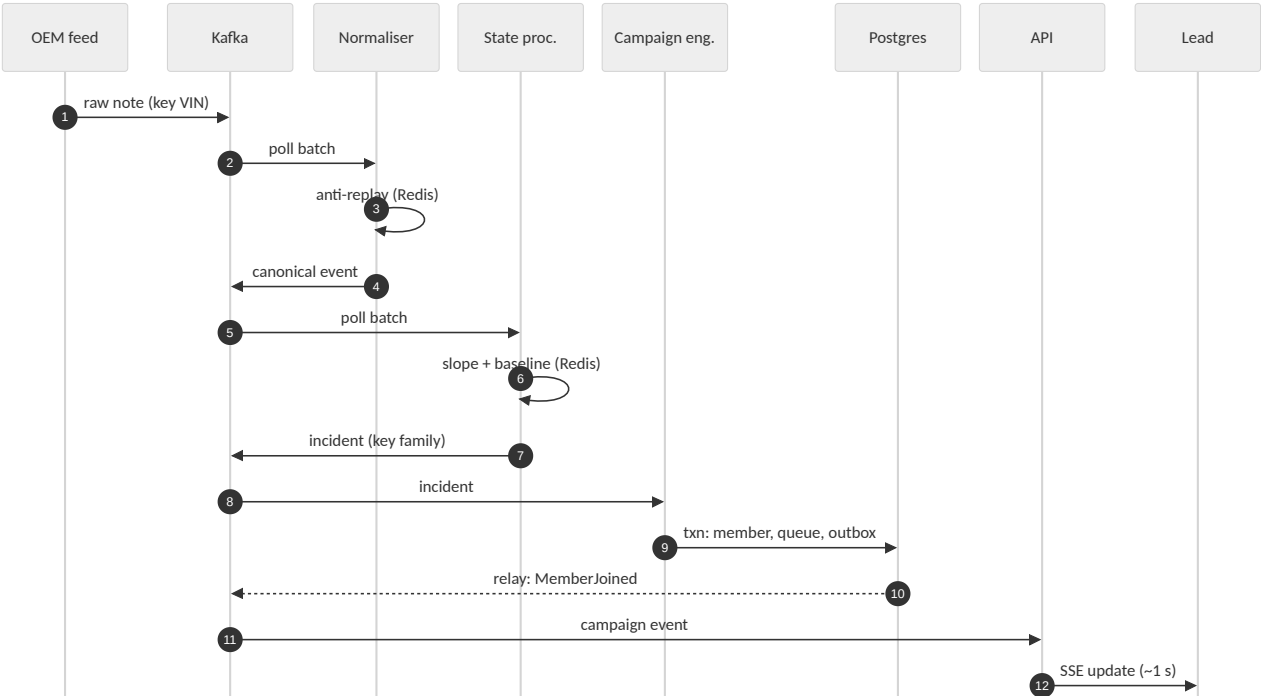


Figure 6 — Happy path: a 13th sister joins an open campaign.

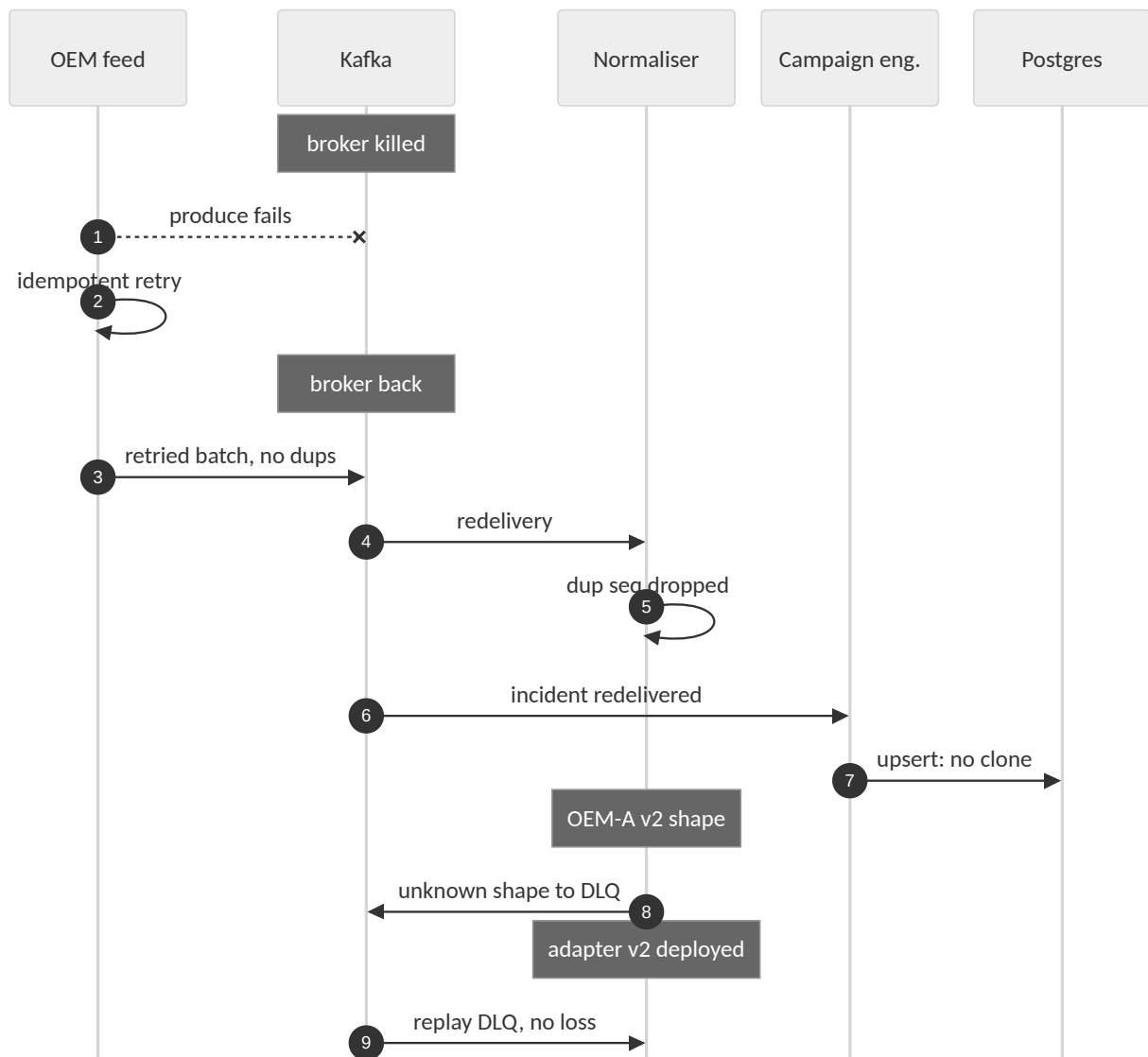


Figure 7 — Failure paths: broker outage, redelivery, duplicate, OEM format change.

14.3 UI

- **Board:** campaign cards (fault family icon, size, depot, trend arrow, firmware chip, severity bar), “watching” strip for candidates, queue side panel, proposal cards (Approve / Ignore with dry-run numbers), activity feed, board-lag badge.
- **Campaign detail:** member table (VIN, joined, clue summary, at-risk badge), shared-context box, similar past campaigns with root causes, override history.
- **Vehicle vs own normal:** line chart with baseline band (median $\pm$ 3 MAD), slope annotation, incident markers; map is not a feature — only depot and geohash5 cell text.
- **Accessibility:** colour is never the only signal (icons + text), keyboard navigable, dark/light themes.

15. Security and compliance

15.1 STRIDE threat model (ingestion path, API, agent)

Threat	Example	Control
Spoofing	Fake OEM feed injects events; stolen user token	mTLS/SASL-SCRAM on OEM connectors (documented; SASL in ha profile); OIDC + short-lived JWT (5 min) + refresh; audience/issuer checks
Tampering	Altered events in transit; audit edits	TLS 1.3; schema validation; audit hash chain + UPDATE/DELETE revoked; object-lock (WORM) bucket for audit in prod
Repudiation	"I didn't approve that"	Every approval/override bound to user sub , IP, version, idempotency key; agent actions under distinct service identity
Information disclosure	Viewer sees driver movement; cross-tenant read (BOLA)	RBAC + Postgres RLS on tenant_id; location masking (geohash5); purpose-bound precise access logged; PII columns encrypted (pgcrypto); least-privilege DB roles
Denial of service	Replay flood, burst, API scraping	Kafka quotas; consumer back-pressure; API rate limits and max page size; payload size limits; DLQ for poison messages
Elevation of privilege	Agent approves its own proposal; prompt injection	Agent scopes exclude approve; human-only endpoint; no free text in evidence; allow-listed tools; grounding check

15.2 Authentication, authorisation, tenancy

Role	Can	Cannot
reliability_lead	All campaign reads, overrides, approve proposals, precise location with purpose	Audit edits, erasure
workshop_planner	Queue edits, campaign reads	Approve rude actions, precise location
viewer (depot operator)	Board and campaign summaries for own depots	Raw trails, driver IDs, exports
agent (service)	Read, simulate, propose, polite writes	Approve, override, see PII
privacy_admin / auditor	Erasure workflow / audit search	Operational actions

Tenant id comes from the token claim and is set per transaction (`SET LOCAL app.tenant_id`) so RLS applies even if an endpoint forgets a filter. Integration tests attempt cross-tenant reads and must get 404.

15.3 Encryption and secrets

- In transit: TLS 1.3 at the ingress (Traefik/NGINX in compose with a local CA via mkcert; cert-manager in Kubernetes); Kafka TLS in ha profile.
- At rest: AES-256 volume/bucket encryption in cloud (KMS-managed keys: EBS/RDS/S3 SSE-KMS, equivalents on GCP/Azure); MinIO SSE locally; column-level pgcrypto for driver PII.
- Secrets: `.env` (git-ignored) with `.env.example` for local; HashiCorp Vault (dev mode container, **COULD**) or External Secrets Operator in Kubernetes; gitleaks in CI.

15.4 Privacy (GDPR, India DPDP Act 2023 / DPDP Rules 2025)

Principle	Implementation
Minimisation	Campaign logic needs depot and cohort, not trails: stream keeps geohash5 for context; precise lat/lon only in the lake for 90 days

Purpose limitation	Precise-location API requires a <code>purpose</code> field, logged in audit
Pseudonymisation	Drivers are pseudonymous IDs; mapping table encrypted and separately permissioned
Retention	Kafka 3 d; Timescale 14 d; lake 90 d warm → archive → delete; audit 7 y (configurable)
Right to erasure	Request → Postgres pseudonym tombstone + Redis purge + Timescale delete + Spark J6 partition rewrite → verification query → audit “erasure completed”; aggregates unaffected
Access logging	Every read of a vehicle page, member list or export writes an audit line (actor, purpose, resource)

15.5 OWASP API Top 10 mapping (abridged)

API1 BOLA → RLS + ownership checks · API2 broken auth → OIDC, short tokens · API3 property-level → response DTOs per role · API4 resource consumption → rate limit, page caps · API5 function-level auth → role decorators, approve endpoint human-only · API6 sensitive flows → idempotency keys on approvals · API7 SSRF → no user-supplied URLs · API8 misconfiguration → hardened images, ZAP baseline · API9 inventory → OpenAPI/AsyncAPI published · API10 unsafe consumption → LLM output validated (grounding check).

16. Observability

Signal	What	Tooling
Metrics	ingest rate per topic; consumer lag per group; DLQ rate by error; dedup drops; incidents/min; campaigns open; event→board latency histogram ; API p50/p95/p99; Redis/DB latency; agent runs and outcomes	Prometheus (OTel SDK + exporters, Redpanda metrics endpoint), Grafana
Logs	Structured JSON with <code>trace_id</code> , <code>vin_hash</code> (never raw VIN in logs for viewers' data), <code>campaign_id</code>	Loki
Traces	Trace context propagated in Kafka headers from normaliser → state → campaign → API; sampled 1%, 100% for incidents	Tempo
Alerts	Lag > 30 s; board lag p95 > 5 s; DLQ spike; unknown OEM shape; outbox backlog; error rate > 1%	Grafana alerting

Troubleshooting walk-through: “board latency spiked to 9 s”

1. Grafana: event→board p95 up; which hop? Consumer lag panel shows `cg.campaign` growing, others flat.
2. Traces for slow incidents: span `campaign.txn` 1.8 s; child span `at_risk_search` dominates.
3. Logs filtered by that trace: large cohort (2,300 vans) after a depot merge; query plan shows seq scan.
4. Fix: add missing composite index / cap at-risk recompute frequency; verify lag drains and p95 returns < 2 s. Record as a post-mortem note in `docs/`.

17. Testing strategy

The brief weights testing heavily; every suite runs in CI on every push (long load/soak runs are manual jobs with committed reports).

Type	Tools	What is covered	Evidence
Unit (≥ 80% on core)	pytest, pytest-cov, Hypothesis (property tests)	VIN check digit; DTC parsing; adapters (v1/v2/Kestrel); anti-replay window (random permutations with dups → exactly-once set); EW slope vs numpy polyfit; robust z; Poisson guard; union-find invariants; family key; queue ordering (D3 below S1); lifecycle state machine; grounding checker	Coverage HTML + badge
Integration	Testcontainers (Redpanda, timescaledb-ha, Redis Stack, MinIO)	Normaliser end-to-end with real broker; outbox → Kafka; RLS cross-tenant denial; member uniqueness under concurrent consumers; Spark J2 on a small parquet fixture	JUnit XML in CI
Contract	Pact (HTTP web ↔ api; message pacts campaign-engine→api/agent); Schema Registry compatibility check	DTO shapes; Avro BACKWARD compatibility gate on schema changes	Pact broker file / CI step
Acceptance (BDD)	behave (Gherkin)	The nine “done” scenarios (§4.4) against the compose stack with a fast scenario seed	behave report
Evaluation	J4 small mode	S1 recall ≥ 0.9, precision ≥ 0.95, decoys admitted = 0, heatwave campaigns = 0	<code>docs/evaluation.md</code>
Performance / load	Custom producer bench (librdkafka) + k6 (API) + Redpanda/consumer-lag metrics	Sustained ingest (target 100K ev/s; report actual), 3× burst for 5 min with zero loss (count in = count out), API p95/p99, 1-hour soak at demo rate (memory, lag stable)	Grafana snapshots + k6 summary JSON
Security	Semgrep , Bandit (SAST); OWASP ZAP baseline (DAST) against compose; pip-audit/npm audit; Trivy image + IaC scan; gitleaks	No critical/high unresolved; findings triaged in <code>docs/security/</code>	SARIF uploaded to GitHub
Compliance	pytest + SQL checks	Every campaign/vehicle read creates an audit row; hash chain verifies; erasure removes VIN from Postgres/Redis/Timescale/lake and totals unchanged	Test report
Chaos	Scripted <code>docker kill / kubectl delete pod</code> (Pumba optional)	Kill broker (ha profile) → no loss; kill campaign-engine mid-batch → no clone, resumes; kill Redis → degraded but correct; kill LLM → template summaries	Recorded run + counts

17.1 Sample BDD scenario

Feature: Cohort campaigns

Scenario: A new sister joins the existing campaign

Given the S1 cooling plant is running at depot "D-042"

And a campaign for "COOLING|D5|URBAN|D-042" is open with 12 members

When vehicle "1AXVN2E47RA000123" crosses its own coolant line

Then the campaign has 13 members within 5 seconds

And no second campaign exists for that family key

And the member's clues mention "slope" and "D-042"

Scenario: Duplicate note does not clone a member

Given the campaign has 13 members

When the same note (VIN, seq) is delivered 3 more times

Then the campaign still has 13 members

17.2 Edge cases explicitly under test

Duplicates; out-of-order; very late (beyond window); reconnect burst; malformed JSON; bad VIN; unknown DTC; unit change; OEM shape change; impossible slope; heatwave; scattered decoy; two outbreaks same week; depot transfer; gentle-driving sister; override survives next tick; two users click at once (409); agent re-proposal after ignore; cross-tenant read; viewer masking; erasure; broker/pod kill.

18. DevOps and one-command setup

18.1 Repository layout

```
cohortwatch/
├── README.md                # problem, architecture image, quick start, env vars, tests, known issues
├── docker-compose.yml       # ONE command: docker compose up
├── .env.example  Makefile  SBOM.spdx.json
├── services/
│   ├── simulator/          # registry, stream, scenarios, noise, ground truth
│   ├── normaliser/         # adapters/, pipeline.py, dlq.py
│   ├── state_processor/     # slope, baseline, context, incidents
│   ├── campaign_engine/     # family, poisson, unionfind, queue, outbox
│   ├── outbox_relay/        telemetry_writer/ lake_archiver/
│   ├── api/                # FastAPI: routers/, auth/, readmodel/, sse.py
│   ├── agent/              # triggers, tools, policy, grounding
│   └── web/                 # React + Vite
├── libs/
│   ├── domain/             # pure business rules (hexagonal core)
│   └── common/             # kafka, schemas (Avro), resilience, otel
├── batch/spark_jobs/        # j0_history ... j6_erasure
├── schemas/avro/            schemas/openapi.yaml docs/asyncapi.yaml
├── db/migrations/ (Alembic) db/seed/
├── infra/
│   ├── compose/ (keycloak realm, grafana dashboards, prometheus, otel, loki, tempo configs)
│   ├── helm/cohortwatch/    # umbrella chart, values-aws.yaml, values-gcp.yaml
│   ├── terraform/aws/       # VPC, EKS, S3+KMS, IAM (gcp/ planned)
│   └── spark/Dockerfile     # apache/spark + hadoop-aws jars
├── tests/ unit/ integration/ contract/ bdd/ load/ chaos/ security/
└── docs/ adr/ sql/ evaluation.md algorithms.md threat-model.md demo-script.md
```

18.2 docker compose — one command

`docker compose up` starts the whole stack, creates topics, migrates and seeds the 100K-vehicle registry, generates history, builds baselines and starts the simulator in `demo` mode. Init jobs are idempotent (they skip if a done-marker exists), so later starts are fast. First boot $\approx$ 10–15 minutes on an 8-core / 16 GB laptop (documented in README); the board is usable as soon as `seed-baselines` completes.

```
name: cohortwatch
x-app: &app
  env_file: .env
  restart: unless-stopped
  depends_on:
    init-topics: { condition: service_completed_successfully }
    migrate:     { condition: service_completed_successfully }

services:
  redpanda:
    image: docker.redpanda.com/redpandadata/redpanda:v24.2.7 # pin exact tags
    command: [redpanda, start, --mode, dev-container, --smp, "2", --memory, 2G,
      --kafka-addr, "internal://0.0.0.0:9092,external://0.0.0.0:19092",
      --advertise-kafka-addr, "internal://redpanda:9092,external://localhost:19092",
      --schema-registry-addr, "0.0.0.0:8081"]
    ports: ["19092:19092", "18081:8081"]
    healthcheck: { test: ["CMD", "rpk", "cluster", "health", "--exit-when-healthy"], interval: 5s,
      retries: 30 }
    volumes: [redpanda:/var/lib/redpanda/data]
  console:
    image: docker.redpanda.com/redpandadata/console:v2.7.2
    environment: { KAFKA_BROKERS: redpanda:9092 }
    ports: ["8088:8080"]
    depends_on: { redpanda: { condition: service_healthy } }
```

```

postgres:
  image: timescale/timescaledb-ha:pg16 # includes TimescaleDB + pgvector
  environment: { POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}, POSTGRES_DB: cohortwatch }
  healthcheck: { test: ["CMD-SHELL", "pg_isready -U postgres"], interval: 5s, retries: 30 }
  volumes: [pg:/home/postgres/pgdata]
redis:
  image: redis/redis-stack-server:7.4.0-v1 # RedisBloom: Bloom, CMS, Top-K, t-digest
  healthcheck: { test: ["CMD", "redis-cli", "ping"], interval: 5s }
minio:
  image: minio/minio:${MINIO_TAG} # pin a release; any S3-compatible store works
  command: server /data --console-address ":9001"
  environment: { MINIO_ROOT_USER: ${S3_ACCESS_KEY}, MINIO_ROOT_PASSWORD: ${S3_SECRET_KEY} }
  ports: ["9001:9001"]
  volumes: [lake:/data]
keycloak:
  image: quay.io/keycloak/keycloak:26.0
  command: [start-dev, --import-realm]
  volumes: [./infra/compose/keycloak:/opt/keycloak/data/import:ro]
  ports: ["8180:8080"]

# ----- one-shot init jobs -----
init-topics:
  image: docker.redpanda.com/redpandadata/redpanda:v24.2.7
  entrypoint: ["/bin/sh", "/scripts/create-topics.sh"] # rpk topic create ... (partitions,
retention)
  volumes: [./infra/compose/scripts:/scripts:ro]
  depends_on: { redpanda: { condition: service_healthy } }
migrate:
  build: ./services/api
  command: ["alembic", "upgrade", "head"]
  env_file: .env
  depends_on: { postgres: { condition: service_healthy } }
seed-registry:
  build: ./services/simulator
  command: ["python", "-m", "simulator.seed", "--vehicles", "100000", "--tenants", "2"]
  env_file: .env
  depends_on: { migrate: { condition: service_completed_successfully } }
seed-history:
  build: ./infra/spark
  command: ["spark-submit", "/jobs/j0_history.py", "--days", "7", "--every-min", "15"]
  env_file: .env
  depends_on: { seed-registry: { condition: service_completed_successfully }, minio: { condition:
service_started } }
seed-baselines:
  build: ./infra/spark
  command: ["sh", "-c", "spark-submit /jobs/j2_baselines.py && spark-submit /jobs/j3_fault_cube.py &&
spark-submit /jobs/j5_signatures.py"]
  env_file: .env
  depends_on: { seed-history: { condition: service_completed_successfully } }

# ----- application -----
normaliser: { <<: *app, build: ./services/normaliser, deploy: { replicas: 2 } }
state-processor: { <<: *app, build: ./services/state_processor, deploy: { replicas: 2 } }
campaign-engine: { <<: *app, build: ./services/campaign_engine }
outbox-relay: { <<: *app, build: ./services/outbox_relay }
telemetry-writer: { <<: *app, build: ./services/telemetry_writer }
lake-archiver: { <<: *app, build: ./services/lake_archiver }
api: { <<: *app, build: ./services/api, ports: ["8000:8000"] }
agent: { <<: *app, build: ./services/agent } # AGENT_ENABLED, LLM_PROVIDER
optional
web: { build: ./services/web, ports: ["3000:80"], depends_on: [api] }
simulator:
  <<: *app
  build: ./services/simulator
  command: ["python", "-m", "simulator.run", "--mode", "demo", "--scenario", "s1,s2,decoys"]
  depends_on: { seed-baselines: { condition: service_completed_successfully } }

# ----- batch (scheduled) -----
spark-master: { build: ./infra/spark, command: ["/opt/spark/sbin/start-master.sh"], ports:

```

```

["8090:8080"] }
  spark-worker: { build: ./infra/spark, command: ["/opt/spark/sbin/start-worker.sh", "spark://spark-
master:7077"] }
  scheduler:    { build: ./batch, command: ["supercronic", "/crontab"] }    # J1 hourly, J2-J6 nightly
(sim time)

# ----- observability -----
otel-collector: { image: otel/opentelemetry-collector-contrib:0.111.0, volumes:
[./infra/compose/otel.yaml:/etc/otelcol-contrib/config.yaml:ro] }
prometheus:     { image: prom/prometheus:v2.54.1, volumes:
[./infra/compose/prometheus.yml:/etc/prometheus/prometheus.yml:ro] }
loki:           { image: grafana/loki:3.2.0 }
tempo:          { image: grafana/tempo:2.6.0 }
grafana:        { image: grafana/grafana:11.2.0, ports: ["3001:3000"], volumes:
[./infra/compose/grafana:/etc/grafana/provisioning:ro] }

# ----- optional profiles -----
k6:             { image: grafana/k6:0.54.0, profiles: [load], volumes: [./tests/load:/scripts] }
bench-producer: { build: ./services/simulator, profiles: [load], command: ["python", "-m",
"simulator.run", "--mode", "bench"] }
# profile "ha": redpanda-1.3 with RF=3 for the broker-kill chaos test

volumes: { redpanda: {}, pg: {}, lake: {} }

```

Image tags shown are examples to pin at build time. `make up = docker compose up -d`; `make demo` resets the scenario clock; `make test`, `make load (--profile load)`, `make chaos`, `make history-billion`, `make eval`. Memory budget ≈ 10–12 GB with everything on; a `lite` override file drops observability and Spark workers after seeding.

18.3 CI/CD (GitHub Actions, every push)

1. **Lint & types**: ruff, mypy, eslint, hadolint, yamllint, terraform fmt/validate, helm lint.
2. **Unit** + coverage gate (≥ 80% on `libs/domain`, `normaliser`, `state`, `campaign engine`).
3. **Build** images (buildx cache) → **Trivy** image + IaC scan → **SBOM** (syft).
4. **SAST**: Semgrep, Bandit; dependency audit; gitleaks.
5. **Integration** (Testcontainers) and **contract** (Pact, schema compatibility).
6. **System**: `compose up (lite, fast scenario seed)` → `behave BDD` → `evaluation (small)` → **ZAP baseline** → compliance tests.
7. Upload reports as artifacts (coverage, junit, SARIF, eval, ZAP HTML). Manual workflow: `load`, `soak`, `chaos (ha profile)` with summaries committed to `docs/perf/`. Release: `tag v1.0-submission` → push images to GHCR.

18.4 Kubernetes, Helm and Terraform (cloud-agnostic)

- **Helm umbrella chart**: one Deployment per service; HPA for API; **KEDA** ScaledObjects on Kafka consumer lag for `normaliser/state/campaign`; `PodDisruptionBudgets`; `NetworkPolicies` (only API exposed via Ingress); `ConfigMaps` from the same env `contract`; `ExternalSecrets` for credentials.
- **Data services in-cluster** (portable): `Redpanda Helm chart` (or `Strimzi Kafka`), `CloudNativePG` with the `Timescale` image, `Redis`, `Spark` via `spark-operator`. Managed equivalents (`MSK`, `RDS/Cloud SQL`, `ElastiCache`) are value-file switches, not code changes.
- **Terraform (AWS first)**: `VPC`, `EKS`, `S3 lake bucket (SSE-KMS, lifecycle rules for warm/cold/delete, object lock for audit)`, `KMS keys`, `IAM roles` for service accounts. `infra/terraform/gcp` mirrors it (`GKE`, `GCS`) — **COULD**; the same Helm chart runs with `values-gcp.yaml`.
- **Cloud-agnostic proof**: the only cloud-specific inputs are the object-store endpoint/credentials and storage class; run the chart on `kind/k3d` locally in CI to prove it.

19. Edge-case catalogue

#	Area	Edge case	Handling
1	Ingestion	Exact duplicate note (same VIN, seq)	Anti-replay window; unique constraints downstream
2	Ingestion	Out-of-order within minutes	Event-time processing; bitmap accepts late-new seqs
3	Ingestion	Very late (hours) after reconnect	Late path: lake + incident re-evaluation for its own window; never counted in “now”
4	Ingestion	Reconnect burst / shift-start 3× surge	Kafka buffers; consumer back-pressure; lag visible on board; KEDA in k8s
5	Ingestion	Malformed JSON, missing required fields	DLQ with offset + reason; never blocks partition
6	Ingestion	Invalid VIN (I/O/Q, bad check digit)	Reject to DLQ; metric by OEM
7	Ingestion	OEM changes payload shape/units overnight	Shape fingerprint → DLQ + alert; versioned adapter; replay DLQ
8	Ingestion	Unknown but valid DTC	Family OTHER; kept, counted
9	Ingestion	Clock skew / future timestamps	Clamp: event_ts > ingest_ts + 2 min → flagged, use ingest time for windowing
10	Ingestion	Seq counter reset (ECU reboot)	Detect large backward jump with newer event_ts → reset anti-replay window for VIN
11	Signal	Impossible values (SoC 140%, negative speed, odometer backwards)	Range validation; field nulled + quality flag
12	Signal	Charge % rising while driving; sensor spikes	Impossible-slope guard; point skipped
13	Signal	One noisy tick	k-of-n confirmation window
14	Baseline	Baseline keeps learning during the outbreak (dying looks normal)	Healthy-window-only training; freeze when < 5 healthy days
15	Baseline	New vehicle with no history (cold start)	Use model × duty cohort baseline until 5 healthy days; clue says “cohort baseline”
16	Baseline	Seasonal drift / heatwave learned as normal	Peer-context adjustment; drift check flags for review
17	Baseline	After repair, vehicle has a new normal	Repair event (manual/queue done) → baseline reset window
18	Family key	Too wide → whole city is one campaign	Place band in key; Poisson guard scales with cohort size
19	Family key	Too narrow → sisters never meet	Behaviour and weather not in key; windows merge via union-find
20	Family key	Behaviour in the key knocks a gently driven sick van out	Behaviour only as clue/queue weight (D6)
21	Family key	Two real outbreaks, same family, same week, two depots	Never merge across place bands (D7)
22	Family key	Same code, different worlds (false family)	Key includes model, duty, place (D1)
23	Family key	Linehaul vans share roads, not yards	Corridor band instead of depot for linehaul duty
24	Firmware	Half the fleet on the same version → giant campaign	Firmware is a clue (“updated ≤ 3 days before onset”), not the default key
25	Firmware	Outbreak spread across depots, shared only by a fresh update	Firmware family type (stretch) with the same Poisson guard
26	Time	Wall clock used for a late note → joins tomorrow’s campaign	Event time for all windows
27	Time	Hour/day buckets split one outbreak	Union-find merge of touching windows

28	Time	Simulated clock vs wall clock in latency metrics	Separate fields; latency always wall time
29	Counting	Two incidents for one van in one window	UNIQUE(vin, family, window); one member
30	Counting	Consumer crash before commit → incident redelivered	Idempotent upsert; outbox event ids
31	Counting	Van transferred depot mid-story	Membership kept; incident snapshot of depot; assignment history table (D5)
32	Counting	Vehicle decommissioned / sold	Status change removes from cohort N and queue; history kept
33	Place	Crowding treated as a cause	Crowding is context/clue only; never a key
34	Place	Packed yard all day makes everything one campaign	Banded context (shift, weather) and peer adjustment, not raw headcount
35	Queue	Loud lonely van jumps the queue	Campaign factor + slope weights (D3 test)
36	Queue	Same van repeatedly pulled off route	Fairness rotation among equal ranks
37	Queue	Workshop capacity exceeded	QueueOverload trigger → agent proposes cross-depot bays (needs approval)
38	Human	Override flipped back on the next tick	Sticky override + versioning; agent never reverts humans
39	Human	Two users act on the same campaign at once	Optimistic concurrency (409, re-read)
40	Human	Every event becomes a card → agent switched off	Triggers only on campaign-level changes; proposal budget
41	Human	Clues that are only a score ("0.91")	Phrase-based clues with numbers and units
42	Agent	Re-proposes the same action after Ignore	Only if materially worse
43	Agent	LLM hallucinates a VIN or number	Grounding check → template fallback
44	Agent	Prompt injection via payload text	No free text in canonical events; data-only evidence block
45	Agent	Agent or LLM down	Board and manual actions unaffected
46	Privacy	Viewer sees raw trails	Masking + role DTOs + audit
47	Privacy	Erasure changes monthly totals	Aggregates hold no personal data; erasure rewrites detail only
48	Privacy	Cross-tenant data leak	RLS on every tenant table; negative tests
49	Infra	Broker/pod killed	Idempotent producer, at-least-once + idempotent effects; chaos test
50	Infra	Redis lost	State rebuildable from Kafka replay; dedup falls back to DB constraints
51	Infra	Outbox relay stuck	Backlog alert; board shows lag; relay leader lock failover
52	Honesty	Claiming 100K ev/s when only 100K IDs were created	Report measured throughput with hardware; separate "load proof" from "product quality"
53	Honesty	Evaluation leaks ground truth into detection	Labels readable only by eval/test code; enforced by DB/S3 permissions

20. NFR targets and honesty rules

NFR	Target (brief)	How we measure	Honest expectation (solo, laptop)
Ingest throughput	100K+ ev/s; survive 3× burst for 5 min, no loss	bench-producer + consumer-lag metrics; count-in = count-out check	Report the measured number and hardware; show horizontal scaling curve (1→2→4 consumers) and extrapolate with partitions, labelled as extrapolation

End-to-end latency	Dashboard < 2 s; critical alert < 5 s	event→SSE histogram	Achievable at demo rate; report p95 under burst too
API latency	p95 < 200 ms, p99 < 500 ms	k6, 200 VUs, board + detail + queue mix	Achievable with read model + indexes
Scalability	Stateless, add nodes without code change	Scale replicas in compose / HPA in kind	Demonstrated for consumers and API
Availability	99.9%, no single point of failure	ha profile: 3 brokers; kill tests	Compose is single-node for Postgres/Redis — stated; Helm values provide replicas
Security	OIDC/JWT, RBAC, tenant isolation, TLS, AES-256, vault, OWASP	Tests + scans	mTLS for OEM connectors documented, not built
Compliance	Audit for every access and agent action; masking; retention; erasure	Compliance test suite	Built

Honesty rules for the submission

(1) Keep load proof separate from planner/detector quality. (2) Every number in the document links to a report file. (3) Features not built are “Partial” or “Planned”, never “Done”. (4) Ground truth never touches detection code. (5) State the hardware for every benchmark.

21. 3-day build plan and feature list

21.1 Day-by-day (one person, AI-assisted coding declared)

When	Build	End-of-block proof
Day 0 (tonight, 1 h)	Repo skeleton, compose with infra only, Avro schemas, family-key spec, document stubs for every template section	<code>docker compose up</code> brings infra green
Day 1 AM	Simulator: registry (100K, check-digit VINs), healthy signal model, S1 plant, decoys D1–D4, mess injection, two OEM formats, ground truth	Sisters visible by eye in a raw query
Day 1 PM	Normaliser (adapters, validation, dedup, DLQ) → state-processor (EW slope, robust z, context adjust, incidents) → telemetry-writer; seed baselines (DuckDB path first, Spark J2 next); unit tests for domain	Incidents for S1 sisters, none for D2/D4; coverage ≥ 80% on domain
Day 2 AM	Campaign engine (family key, Poisson guard, union-find, membership, at-risk, queue, outbox) + relay; API (OIDC, RBAC, RLS, pagination, rate limit, audit, SSE)	Done-list items 1–6 pass as BDD
Day 2 PM	Web (board, detail, vehicle page); agent (triggers, tools, dry-run, proposals, grounding, template mode); pgvector signatures; Spark J0/J2/J3/J4 in compose; CI pipeline (lint, unit, Trivy, Semgrep, Testcontainers)	Full demo story end-to-end in the UI
Day 3 AM	Evidence: bench + burst + k6 + soak (1 h in background), chaos kills, ZAP baseline, EXPLAIN ANALYZE before/after, evaluation report, erasure test; Helm chart + Terraform (validate, not applied)	Numbers for NFR, test and SQL tables
Day 3 PM	Solution document (diagrams, 3–5 ADRs, STRIDE), README, SBOM, declarations; record 5-min video (script §22); tag <code>v1.0-submission</code>	Submission

21.2 Feature list (template §4)

ID	Feature	User story	Priority	Code path
F-01	100K-vehicle simulator with plants, decoys, mess, ground truth	As an evaluator I want a reproducible messy fleet so that claims can be scored	MUST	services/simulator
F-02	Multi-OEM normalisation + DLQ + replay	As a platform owner I want format changes absorbed without data loss	SHOULD	services/normaliser
F-03	Idempotent dedup & late-data handling	As a lead I want duplicates never to clone members	MUST	normaliser, libs/domain
F-04	Per-vehicle baseline (level + slope) with peer context	As a lead I want a van judged against its own normal, not a global line	MUST	state_processor, batch/j2
F-05	Incident with plain-language clues	As a lead I want to know why a van is flagged	MUST	state_processor/clues.py
F-06	Campaign formation (family key, Poisson guard, union-find)	As a lead I want one outbreak card instead of 15 tickets	MUST	campaign_engine
F-07	At-risk sisters	As a planner I want to inspect vans before they fail	SHOULD	campaign_engine/at_risk.py
F-08	Workshop queue with reasons and cost of waiting	As a planner I want the next bay to go to the most consequential van	MUST	campaign_engine/queue.py
F-09	Campaign board + detail + vehicle page (SSE live)	As a lead I want to see outbreaks form in seconds	MUST	services/web, api
F-10	Override “not an outbreak” (sticky, versioned)	As a lead I want my judgement to stick	MUST	api/routers/campaigns.py

F-11	Agent proposals with approval, dry-run, grounding, audit	As a lead I want drafted actions but final say on disruptive ones	SHOULD	services/agent
F-12	Similar past campaigns (pgvector)	As a lead I want to know if we have seen this before	SHOULD	batch/j5, api/similar.py
F-13	Firmware clue	As a lead I want to know if a recent update is shared	SHOULD	campaign_engine/clues.py
F-14	Spark batch: history, baselines, fault cube, back-test	As an analyst I want fleet-wide rates and proof of accuracy	MUST	batch/spark_jobs
F-15	OIDC, RBAC, RLS, masking, audit hash chain	As a compliance officer I want least-privilege access and evidence	MUST	api/auth, db/migrations
F-16	Right-to-erasure workflow	As a data subject I want my data removed without breaking totals	SHOULD	api/privacy, batch/j6
F-17	Observability dashboards and alerts	As an operator I want to find why the board is late	SHOULD	infra/compose/grafana
F-18	Firmware-keyed campaigns (S2 across depots)	As a lead I want cross-depot software outbreaks grouped	COULD	libs/domain/family.py
F-19	OEM campaign report export	As an OEM quality engineer I want a field report	COULD	api/export.py
F-20	Family discovery (k-means)	As an analyst I want suggested new fault families	COULD	batch/j5
F-21	Map view, driver scoring, OTA rollout control	—	WON'T	—

21.3 Cut order if time runs out

Cut from the bottom: F-20 → F-19 → F-18 → Terraform GCP → F-16 batch rewrite (keep DB erasure) → LLM mode (keep template agent) → Spark cluster in compose (run jobs in local mode) → F-12. **Never cut:** F-01, F-03, F-04, F-06, F-08, F-09, tests for them, the evaluation numbers, and the one-command setup.

22. Demo video script (≤ 5:00)

Time	Segment	Show
0:00–0:30	Problem	“Workshops drown in single warnings; shared failures take ~7 weeks to name.” Show a wall of 4,000 raw alerts vs one question: <i>is this group getting the same illness?</i>
0:30–1:00	Solution	One-liner; three objects: vehicle vs its own normal → campaign → queue.
1:00–3:00	Live demo	<code>docker compose up</code> already running; board fills; S1 campaign opens at D-042 (12 members) with clues (slope, place); heatwave region shows no campaign; decoy with same code stays out; 13th sister joins live (count ticks, no new card); at-risk list; queue puts sisters above the loud lonely van; agent card “inspect 13 + 3 at-risk tomorrow; pull 2 off route?” → Approve → audit line; override demo; viewer login shows masked data.
3:00–4:15	Under the hood	Architecture diagram (10 s); Grafana: ingest rate, lag, event→board p95; kill a broker or campaign-engine → recovers, member count unchanged; Spark back-test output; CI pipeline green.
4:15–5:00	Impact & next	Precision/recall and lead time vs baselines; measured throughput stated honestly; next steps (pilot with real OEM feed, firmware-keyed families, OEM report).

23. ADRs, risks and future work

23.1 Architecture Decision Records (summaries)

ADR	Context → Decision → Consequences
ADR-1 Messaging	Need replayable, partitioned, ordered streams at 100K ev/s with one-command setup. Options: Apache Kafka, Redpanda, RabbitMQ, MQTT broker. Redpanda (Kafka API) . + replay, registry built-in, single binary; – vendor flavour (mitigated: pure Kafka clients, portable).
ADR-2 Stores	Relational integrity + time-series + vectors with minimal ops. Options: Postgres+Timescale+pgvector; Postgres+Cassandra+Qdrant. One Postgres engine (3 roles) + Redis + S3 . + fewer systems to secure, joins with RLS; – shared resource contention (mitigated: separate schemas/pools, Timescale compression, replica for reads); Qdrant swappable via port.
ADR-3 Single writer per family	Two engines could open duplicate campaigns concurrently. Options: distributed lock, DB advisory lock, partition by family key. Key incidents by family key . + no locks, natural ordering; – big family = one partition (fine at incident volume).
ADR-4 CAP split	Campaign/approval state must never diverge; telemetry must never stop. CP for campaigns, approvals, audit; AP for telemetry/state . + correctness where it matters; – board read-only during DB failover (acceptable, shown in UI).
ADR-5 Batch on S3 not HDFS	\$7 allows HDFS or S3. Spark + Parquet on S3 API (MinIO) . + cloud-agnostic, no NameNode; – eventual listing semantics (use committers/manifest).
ADR-6 Python stream services, not Flink	Solo, 3 days; state is O(1) per VIN. Consumer-group services + Redis state . + speed of build; – fewer built-in windowing features. Revisit when state per key grows or exactly-once joins are needed.
ADR-7 Rules group, model scores	Explainability vs flexibility. Rule-based family key + statistical guard; ML only for self-abnormality and at-risk . + defensible clues; – may miss unforeseen families (mitigated by k-means discovery, advisory).

23.2 Risks and technical debt

Risk	Impact	Mitigation
Scope overrun (solo, 3 days)	Unfinished core	MoSCoW + cut order; done-list first; documents stubbed on Day 0
Laptop cannot reach 100K ev/s	NFR shortfall	Honest measured figure + scaling curve + partition math
Family-key tuning	False or fragmented campaigns	Spec before code; decoy tests; back-test sweep of k_min, α
Simulator realism	Results look circular	Evaluate against baselines on the same data; publish simulator parameters; admit limitation
Compose memory footprint	Demo instability	lite override; Spark jobs in local mode after seeding
Image/tag drift (e.g. registry changes)	Broken one-command setup	Pin digests; CI boots compose weekly

23.3 Future enhancements (prototype → pilot)

1. Connect one real OEM cloud feed (sandbox) and a real workshop calendar; measure detection lead time on historical repair records.
2. Firmware-keyed and supplier-batch families; OEM field-report export aligned with early-warning reporting formats.
3. Move stateful stream logic to Flink or Kafka Streams when multi-signal joins are needed; Iceberg tables for time travel and schema evolution in the lake.

24. Deliverables traceability

Brief requirement	Where satisfied	Artefact
§6 Simulated data ≥ 100K vehicles, own simulator	§6	services/simulator, seeded registry
§6 Real-time + batch analytics	§5, §9, §10	stream services; Spark jobs J0–J6
§6 Relational + NoSQL + vector, justified	§8.1, ADR-2	Postgres/Timescale, Redis, pgvector
§6 Secure APIs + usable web UI	§14, §15	api, web, OpenAPI
§6 Containerised, tested, cloud-agnostic	§17, §18	Dockerfiles, compose, Helm, Terraform, CI
§7 IoT ingestion (bursty, OOO, dup; idempotent; schema-validated; back-pressure)	§6.5, §7, §9.1–9.2	normaliser, Avro registry
§7 Messaging with replay	§7	topics, consumer groups, replay tool
§7 Big data (seconds + billions of rows)	§5.4, §10	latency histogram; 1B-row history mode
§7 ML vs baseline	§11	docs/evaluation.md
§7 Agentic AI with guardrails + audit	§12	services/agent, audit_log
§7 Log analytics	§16	Grafana, Loki, Tempo, Prometheus
§7 DevOps	§18	compose, Helm, Terraform, Actions
§8 3NF, polyglot, partitioning, generator, EXPLAIN ANALYZE	§8	ER diagram, migrations, docs/sql
§9 Algorithms	§9	docs/algorithms.md (pseudocode, complexity, scale tests)
§10 CAP/PACELC, principles, lifecycle, ADRs	§8.6, §13, §23	docs/adr/*.md
§11 NFRs	§20	docs/perf/
§12 Testing (unit, integration, contract, BDD, load, security, compliance, chaos)	§17	CI artefacts
§13 Deliverables: doc, repo + README + one-command + seeded 100K, diagrams, ADRs, demo, test evidence, DevOps pack, STRIDE, algorithms/SQL write-up, video	§18.2, §21, §22	Repository at tag <code>v1.0-submission</code>
§14 Rules: synthetic data, declare OSS and AI tools, code freeze tag, originality	Declarations section	SBOM, AI-tools declaration, fictitious OEM names

Sources

- McKinsey & Company — “When warranty costs rival R&D spend: Remaking vehicle quality with AI”. <https://www.mckinsey.com/industries/automotive-and-assembly/our-insights/when-warranty-costs-rival-r-and-d-spend-remaking-vehicle-quality-with-ai>
- EE Times — “Leverage telematics and on-board diagnostics for warranty management”. <https://www.eetimes.com/leverage-telematics-and-on-board-diagnostics-for-warranty-management/>
- NHTSA — Early Warning Reporting. <https://www.nhtsa.gov/vehicle-manufacturers/early-warning-reporting>
- Journal of Intelligent Information Systems (2022) — “A context-aware unsupervised predictive maintenance solution for fleet management”. <https://link.springer.com/article/10.1007/s10844-022-00744-2>
- Upstream Security — “Leveraging Cohort Analysis for Fleet-Wide Anomaly Detection in Automotive Cybersecurity”. <https://upstream.auto/blog/cohort-analysis-for-fleet-wide-anomaly-detection-in-automotive-cybersecurity/>
- Kozaeva & Bade — “From fault logs to insights: TF-IDF vectorization and cluster-based behavioral analysis of railway vehicles”, J. Rail Transport Planning & Management. <https://www.sciencedirect.com/science/article/pii/S2210970626000454>

- Motorq — “Motorq Launches Fuse to Turn Vehicle Insights into Fleet Cost Savings” (14 Apr 2026). <https://www.prnewswire.com/news-releases/motorq-launches-fuse-to-turn-vehicle-insights-into-fleet-cost-savings-302741092.html>
- Connected Vehicle Intelligence Hackathon — Problem Statement & Case Study; Solution Document Template (provided).

Motorq is used only as an industry reference; OEM names “Aurex” and “Kestrel” and all data are fictitious/synthetic.